IoT-Enabled Smart bus route monitoring and revenue management system

INDIVIDUAL THESIS

Sachintha H D D

IT22264152

Individual component: Policy-Driven Bus Scheduling and Time-Slot Management Engine

Group ID: R26-ISE-004

B.Sc. (Hons) in Information Technology Specialized in Information Technology

Department Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

Oct 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Sachintha H D D

Student ID: IT22264152

Signature: _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Sanjeeva Perera		
Co-supervisor: Buddima Attanayaka		

ABSTRACT

Public bus transportation systems usually run on fixed timetables that're hard to change when conditions shift. Delays, bus breakdowns drivers not being available, and service stops can break the plan and force quick rescheduling. In places where timetable work still needs a lot of effort these problems can cause gaps in service, scheduling clashes and waste of buses and drivers. The overall IoT-enabled smart bus project tackles these problems by combining vehicle tracking, route checking, policy-based scheduling, revenue control and driver monitoring in one system. This research looks at the Policy-Driven Bus Scheduling and Time-Slot Management part.

The study offers a rule-based scheduling system that sorts bus work into Peak and Night slots and changes the plan automatically when problems happen. The component adds a Disruption-Aware Dynamic Schedule Adaptation Algorithm that spots trips that are affected, chooses the time window for changes, filters replacement buses by who is free and can finish their current job, ranks possible buses by how well they fit the route and how much work they already have, and picks a suitable driver. If no suitable bus can be found, the trip is. The reason is logged.

The part is made with Java and Spring Boot. Uses a PostgreSQL database for schedule data.

The test checks how well the sorting works, how the policy rules run, how disruptions are dealt with, how resources are assigned, how consistent the schedule stays, and how cancellations behave in controlled test cases.

The method gives an understandable way to change bus schedules while still following the set rules.

Keywords: Policy-Based Scheduling, Bus Scheduling, Time-Slot Management, Dynamic Rescheduling, Disruption Management.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor and co-supervisor for their valuable guidance, continuous support, and constructive feedback throughout the development of this project. The academic staff of the Faculty of Computing at the Sri Lanka Institute of Information Technology (SLIIT) provided academic support to me during my research work while sharing their knowledge with me, which I appreciate. My appreciation also goes to the Computer System Administrator of the Road Passenger Transport Authority for providing useful information and insights related to the existing transport management practices. My friends and family supported me through this work which I want to express my gratitude.

TABLE OF CONTENTS

1 INTRODUCTION 1

1.1 Overall project context 1

1.2 Individual component context 2

1.3 Problem statement 3

1.4 Research gap 4

1.5 Aim and objectives 5

1.6 Research questions 5

1.7 Scope limitations and assumptions 6

1.8 Individual contribution 6

1.9 Thesis structure 7

2 LITERATURE REVIEW AND RELATED WORK 8

2.1 Review method and scope 8

2.2 Domain and theoretical foundations 8

2.2.1 Public bus scheduling as a multi-stage process 8

2.2.1 Timetabling and vehicle scheduling 9

2.2.2 Time slots and operational state 9

2.2.3 Decision approaches: optimization, heuristics and rules 9

2.3 Related research 10

2.3.1 Vehicle and driver scheduling 10

2.3.2 Dynamic scheduling and real-time operational control 11

2.3.3 Disruption management in public transportation 11

2.3.4 Disruption scope in rescheduling 12

2.3.5 Workload balance in replacement decisions 13

2.3 Existing systems products and techniques 14

2.3.1 Optimization-based techniques 14
2.3.2 Real-time control techniques 14
2.3.3 Rule-based techniques 15
2.3.4 Comparison of techniques 15
2.4 Gap synthesis and conceptual framework 16
2.4.1 Synthesis of the literature 16
2.4.2 Identified research gap 18
2.4.3 Conceptual framework 19
3 RESEARCH METHODOLOGY 21
3.2 Research design 21
3.3 Data participants or study materials 22
3.4 Experimental or study procedure 23
3.5 Measures and evaluation metrics 25
3.6 Data analysis 26
3.7 Ethics privacy security and responsible AI 26
3.8 Reproducibility and research artefacts 27
4 PRODUCT AND SOLUTION DESIGN 29
4.1 Requirements 29
4.1.1 Functional Requirements 29
4.1.2 Non-Functional Requirements 33
4.2 Component architecture within the complete solution 35
4.2.1 Scheduling Component Architecture 36
4.3 Design alternatives and decisions 38
4.1 Detailed design 38
4.1.1 Service Design 38
4.1.2 Monthly Base Scheduling 39

4.1.3 Operational Turn Representation 39
4.1.4 Disruption Handling and Reallocation Window 39
4.1.5 Affected Trip Identification and Preparation-Time Check 40
4.1.6 Candidate Bus Filtering 40
4.1.7 Bus Ranking 40
4.1.8 Reallocation and Cancellation Decision 41
4.1.9 Notification 41
4.1.10 User Interfaces 41
4.2 Standards and professional practice 42
4.2.1 Software Development Standards 42
4.2.2 Security Standards 42
4.2.3 API and Database Standards 42
4.2.4 Logging and Traceability 43
4.2.5 Documentation and Maintenance 43
4.2.6 Responsible System Practice 43
5 IMPLEMENTATION INTEGRATION AND TESTING 44
5.1 Development environment and technology stack 44
5.2 Component implementation 45
5.2.1 Monthly Scheduling Service 45
5.2.2 Turn Management Service 45
5.2.3 Disruption Handling Service 46
5.2.4 Rescheduling 46
5.2.5 Bus Ranking 47
5.2.6 Schedule Update 48
5.2.7 Notification 48
5.3 Integration with the other components 49

5.3.1 Integration Flow 49
5.3.2 API and Service Integration 50
5.3.3 Database Integration 51
5.3.4 Integration with IoT and Hardware Components 52
5.3.5 Interaction with Other Functional Components 53
5.4 Verification and software testing 53
5.4.1 Unit Testing 53
5.4.2 Integration Testing 54
5.4.3 System and End-to-End Testing 54
5.4.4 Test Scenarios 54
5.4.5 User Acceptance and Performance Testing 55
5.4.6 Deployment and Operation 56
5.5 Deployment and operation 56
6 RESULTS AND VALIDATION 58
6.1 Research results 58
6.1.1 Reference Configuration for Monthly Base Scheduling 58
6.1.2 Evaluation Questions 58
6.1.3 Evaluation Measures 59
6.2 Product and Technical Results 60
6.2.1 Monthly Base Scheduling 60
6.2.2 Disruption Handling and Reallocation 61
6.3 User or stakeholder validation 62
6.4 Integration results 62
6.4.1 Monthly Schedule Generation Path 63
6.4.2 Disruption and Reallocation Path 63
6.4.3 Worked Decision Trace 64

6.5 Summary of findings 65
7 DISCUSSION 66
7.1 Interpretation of findings 66
7.1.1 What the Results Say About the Design 66
7.1.2 Reading the Metrics Together 67
7.1.3 Answering the Research Questions 67
7.2 Comparison with literature baselines and existing solutions 68
7.3 Research contribution 70
7.4 Product contribution 71
7.5 Limitations and threats to validity 71
7.5.1 Limitations of the Component 71
7.5.2 Threats to Validity 73
7.6 Implications 73
8 CONCLUSION AND FUTURE DIRECTIONS 75
8.1 Conclusion 75
8.2 Achievement of Objectives 76
8.3 Individual Research and Product Contributions 77
8.3.1 Research Contribution 77
8.3.2 Product Contribution 78
8.3.3 Place in the Overall System 78
8.4 Future directions 79
REFERENCES 80
GLOSSARY 82
APPENDICES 84
Appendix A: Risk Analysis Matrix 84

.

LIST OF FIGURES

Figure 1: Overall System Architecture of the IoT-Enabled Smart Bus Solution..... 35
Figure 2: Architecture of the Policy-Driven Bus Scheduling and Time-Slot Management Engine..... 36

LIST OF TABLES

Table 1: Gap Analysis of Existing Scheduling Approaches against the Proposed Component 18
Table 2: Stages of the Research Workflow 22
Table 3: Operational Data Entities and Required Information 23
Table 4: Functional Requirements of the Scheduling Component 30
Table 5: Major Modules of the Scheduling Component 37
Table 6: Design Alternatives for Replacing an Unavailable Bus 38
Table 7: Technology Stack Used in the Component 44
Table 8: Test Scenarios and Expected Outcomes 55
Table 9: Reference Configuration for Monthly Base Scheduling 58
Table 10: Evaluation Measures and Definitions 59
Table 11: Verification Checks on the Generated Monthly Base Schedule 60
Table 12: Evaluation Results for Disruption Handling and Reallocation 62
Table 13: Worked Decision Trace for Bus Unit B01 Breakdown 64
Table 14: Summary of Findings against the Research Questions 65
Table 15: Answers to Research Questions 68
Table 16: Conceptual Comparison of the Proposed Component with Existing Approaches 69
Table 17: Threats to Validity and Their Handling 73
Table 18: Achievement of Objectives 77

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
CCR	Constraint Compliance Rate
CR	Cancellation Rate
DSR	Design Science Research
FR	Functional Requirement
GPS	Global Positioning System
GTFS	General Transit Feed Specification
HTTP	Hypertext Transfer Protocol
IMU	Inertial Measurement Unit
IoT	Internet of Things
JPA	Java Persistence API
JWT	JSON Web Token
MILR	Mixed-Integer Linear Programming
NFR	Non-Functional Requirement
REST	Representational State Transfer
RQ	Research Question
SLIIT	Sri Lanka Institute of Information Technology

Abbreviation	Meaning
SRR	Successful Reallocation Rate
TSL	Transport Layer Security
UI	User Interface

1 INTRODUCTION

1.1 Overall project context

Public bus transportation is a part of how people move in cities and suburban areas. It offers a cost-effective and easy way for many passengers to travel. In Sri Lanka, both private bus companies and government-run services help carry people around. Keeping these bus services running well depends on control over buses, routes, drivers, operating hours, and schedules.

The research project called IoT-Enabled Smart Bus Route Monitoring and Revenue Management System aims to create a transportation system. This system brings together vehicle data, route tracking, rules-based timetables, fare collection, and driver safety checks. It uses sensors inside the buses, cloud networks, data processing systems, and apps for users. All of this works together to give a shared view of real-time information across the transportation network.

The framework has four main functionalities in total, namely, IoT-based vehicle monitoring and map matching, policy and time-slot management, distance-based revenue management, and driver monitoring. In this architecture, the scheduler component is responsible for translating operational information and transport policies into scheduling decisions.

The project proposal notes that manual timetable preparation and timekeeper-based schedule management are challenges, as it can be difficult to maintain planned schedules when delays, breakdowns, and irregular service conditions occur. So the scheduling part is intended to provide a better systematic mechanism to maintain and adapt bus operations.

1.2 Individual component context

The process consists of two stages. The first step is to generate a planned schedule for bus operations. The schedule determines monthly basic tasks taking into account the number of buses, expected trips, route specifics, and scheduling limitations. The schedule helps an operator to assign buses to their shifts.

The second stage is to manage the operations in real time; if a bus develops a malfunction, if a driver is on leave, or if any other operational incident has occurred, the initial schedule cannot be followed anymore. Therefore, the system must be able to locate the problematic shifts and decide whether those shifts can be covered by other buses.

Thus, the concept of the time slot is used in the context of the operational shift positioned in a schedule and having to be managed when an incident occurs. In this case, the system's task goes beyond just changing the shift name from Peak to Normal. The system must find out if the original operational shift can proceed with the help of other resources.

The proposal emphasizes the need for automated schedule adjustment during incidents like missed trips, delayed travel, and bus unavailability. The system must be able to analyze the situation and the existing scheduling conditions and generate an alternative schedule.

1.3 Problem statement

A monthly bus schedule has a basic assumption of planned buses being reliable for the entire duration of service. In reality, such an assumption can sometimes be thwarted if a bus becomes inoperative after the schedule has been produced.

For instance, if one of the buses earmarked for several future trips breaks down, all other scheduled trips assigned to it can become affected. In an old-fashioned approach to solve the problem, a manager or a scheduler needs to find the affected trips, look for an alternative bus, check its availability at that particular time, check how it was used before and how much time it needed to come back from its previous trip, and to find out if the necessary driver is available.

It is not enough to just choose the first bus available since it may be already assigned, out of service, lack enough turnaround time, run on a different route, be overloaded with work, or have been used too many times before for reallocation; in addition, the regular driver of the bus may be unavailable as well, which requires a second decision-making problem to assign a replacement driver.

The research question posed by the problem is then:

How can a bus scheduling system dynamically adapt a pre-planned schedule when an assigned bus becomes unavailable, while identifying feasible replacement resources and maintaining route compatibility, preparation-time, turnaround, and workload constraints?

1.4 Research gap

The field of public transport scheduling has looked at things like timetable development, vehicle scheduling, driver scheduling, and the management of interruptions. In particular, research into real-time rescheduling has found that it is possible to use operational data to adapt vehicle schedules in response to changes in real-time situations. Lai and Leung, for example, have explored how vehicle and driver scheduling can be done dynamically under conditions where travel times are uncertain and change from one moment to another.

Research into bus operations has established that rescheduling is a technique for dynamic control that helps mitigate the impact resulting from service disruptions. However, these studies are very often based on mathematical modelling, operational and maintenance regularity guarantees, or even passenger-oriented aims.

The research project aims to find a gap in existing knowledge concerning the problem of service disruption, which is treated as a standalone optimization task rather than incorporated into a process-oriented operational workflow. The project suggests creating an operational workflow where an event is recognized, fleet status is determined, available resources are analyzed, trips are allocated, and the schedule is adjusted.

In this study, the author fills in the gap at the component level by combining monthly basic scheduling, operation time slot management, disturbance-based trip identification, constraint-based bus filtering, fairness-based ranking, sequential driver selection, dynamic trip redistribution, and controlled cancellation to form a single framework that transforms planned scheduling into disturbance-aware operational rescheduling.

1.5 Aim and objectives

The purpose of this study is to build and apply a policy-driven approach to bus scheduling and a time-management mechanism that helps produce a structured base schedule and redistribute bus turns affected during normal operations disruptions.

Specific Objectives

1. To develop a system that generates a base schedule on a monthly basis to allocate anticipated bus turns to available buses.
2. To express the generated schedule through manageable operational turns/time slots.
3. To set up rules and restrictions concerning bus availability, bus assignment conflicts, preparation time, and bus turnaround time.
4. To create a system aware of disruptions that would allow anticipating future turns influenced by the non-availability of the buses.
5. To establish a fair ranking system for substitute buses depending on route compatibility and other parameters including workload and previous reallocations.
6. To create a mechanism for selecting the driver so that the regular driver of the selected bus is given priority over other candidates and only when this is impossible, the method of selecting based on workload is used for the situation.

1.6 Research questions

RQ1: What approach can be used to arrange monthly bus operations in order to allow for adjustments based on the occurrence of service disruptions?

RQ2: How can factors such as bus availability, route compatibility, preparation time, conflicts in assignment, and turnaround needs be incorporated into the selection process for replacement buses?

RQ3: How can previous relocations and workload be utilized in replacement bus selection in order to make resource allocation more equitable?

RQ4: To what extent does the suggested algorithm ensure that schedule continuity is established when the scheduled bus is unavailable?

RQ5: When is the use of reallocation necessary for an affected turn, and when should the turn be canceled instead?

1.7 Scope limitations and assumptions

The area of focus for this research is bus scheduling and operation schedule changes.

This component includes monthly route schedule creation, bus assignments, shifts in timetable and transportation slots, as well as emergency time slots and specific schedule changes. In the case of an incident, this component will detect affected trips, filter an eligible bus fleet, select replacements, and give assignments to drivers.

The component does not aim to solve all public transport optimization problems. It does not include prediction of passenger demand, fare calculation, map of routes matching, driver behavior classification, and revenue calculation based on distance.

The algorithm assumes that the system has up-to-date information about bus schedules, drivers, trips, and whether they are affected by an incident or not.

1.8 Individual contribution

The unique aspect of this study is the development of the Policy-Driven Bus Scheduling and Time-Slot Management component, which has been designed and implemented.

The main contribution of the study relates to the Disruption-Aware Dynamic Schedule Adaptation Algorithm. The algorithm improves the initial scheduling process by offering operational assistance in situations when the allocated bus resource is no longer available. The contribution provides an all-encompassing operational framework that includes a monthly base scheduling, time-slot representation, determination of the disruption window, identification of the affected turn, preparation time validation, filtering of the bus candidates, ranking of the buses, selection of drivers, dynamic reallocation of the buses, fallback strategy in case of cancellations, and real-time update of the schedule state.

1.9 Thesis structure

Chapter 1 introduces the research problem, the research gap, and the aim and objectives, followed by the research questions, the scope and assumptions, and the individual contribution.

Chapter 2 reviews prior work on bus scheduling, vehicle scheduling, real-time scheduling, disruption management, time-slot management, and rule-based resource allocation, and closes with the gap synthesis and conceptual framework.

Chapter 3 sets out the research methodology: the research design, study materials, experimental procedure, evaluation metrics, data analysis, and the ethical and reproducibility considerations.

Chapter 4 presents the product and solution design, covering the requirements, the architecture within the complete system, the design alternatives, and the detailed design of each processing stage.

Chapter 5 explains how the component was implemented, integrated with the other components, tested, and deployed.

Chapter 6 reports the experimental results and the validation of monthly base scheduling, disruption handling, and reallocation.

Chapter 7 discusses the findings against the research questions and existing solutions, together with the contributions, limitations, and implications of the work.

Chapter 8 concludes the thesis by checking the outcome against each objective and outlining directions for future research.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

This chapter analyzes literature relevant to a rule-based, event-driven bus scheduling component, focusing on public bus scheduling and related topics such as timetabling, vehicle and driver scheduling, real-time control, disruption management, and event-driven adaptation. The literature review aims to establish theoretical foundations, evaluate existing techniques, and identify gaps addressed by the proposed Disruption-Aware Dynamic Schedule Adaptation Algorithm. It encompasses studies from other transport modes where applicable while confirming exclusions like route network design. The search process details databases, keywords, publication periods, and inclusion criteria used to screen sources, with retained sources organized thematically to compare research focus and limitations, culminating in a gap analysis and conceptual framework.

2.2 Domain and theoretical foundations

2.2.1 Public bus scheduling as a multi-stage process

Delivering a bus service involves a chain of linked decisions: developing the timetable, assigning vehicles to trips, assigning drivers to the resulting duties, and controlling operations on the day. Ibarra-Rojas et al. [1] reviewed this body of work and organised the decisions into strategic, tactical, and operational levels. They noted that real-time control is needed to respond to changes that arise during service. Ceder [2] likewise presents timetable development and vehicle scheduling as connected activities, in which the available fleet is allocated to the timetabled services.

A schedule prepared in advance cannot anticipate every event of the operating day. A vehicle breakdown or the absence of a driver can leave planned assignments unworkable, and this is the situation the proposed component is designed to handle. Scheduling is therefore treated here as continuing after the plan is created. The monthly base schedule provides the initial allocation of trips and buses, while operational rescheduling deals with the time slots affected when an event disrupts that allocation. The two are connected but handled as separate activities.

2.2.1 Timetabling and vehicle scheduling

Timetabling decides when services run, and vehicle scheduling decides which vehicle operates each resulting trip, so the timetable defines the set of trips the fleet must cover. Ceder [2] discusses how the two activities can be integrated and describes scheduling as a way of providing the required service while limiting the number of vehicles used. Conventional vehicle scheduling formulations accordingly take a fixed set of trips and assign them to available vehicles [1], [2].

Reassigning a trip after planning requires more than a list of idle buses. In the proposed system, a bus can take over a trip only if its preceding trip, next available time, route compatibility, and turnaround constraints allow it. For this reason the operational state of every bus must be represented explicitly.

2.2.2 Time slots and operational state

A timetable can be viewed as a set of trips, each tied to departure and arrival times. A bus changes state as it completes one trip and becomes free for the next, and the trips given to a single vehicle must follow one another without timing conflicts [2]. Time and resource allocation therefore cannot be handled independently.

Time-slot management here goes beyond classifying periods as peak, normal, or night. Those categories supply policy context, but the central function is handling the time slot of an affected trip. When a bus becomes unavailable, the trips assigned to it within the disruption window are identified. For each affected slot, the system then checks whether another bus can take it over without conflicting with that bus's existing schedule. Time-slot management is thus treated as an operational resource-allocation problem and not only as a classification of periods.

2.2.3 Decision approaches: optimization, heuristics and rules

Scheduling problems in public transport have been approached through exact optimization, mathematical programming, heuristics, metaheuristics, and rule-based methods. Radmankian et al. [3] reported that mathematical programming, particularly Mixed-Integer Linear Programming (MILP), remains widely used to express discrete scheduling constraints, and that metaheuristics and other methods attract growing

attention because complex scheduling problems are computationally difficult. Exact methods can give strong solutions, but their computational cost may rise as the number of vehicles, trips, constraints, and operational events increases [4].

Rule-based decision-making offers another trade-off. Rules are applied in a fixed order, and the whole schedule is not optimized. The proposed component uses this form because each step of a disruption response can then be traced, and the response is produced directly from the current operational state.

2.3 Related research

2.3.1 Vehicle and driver scheduling

Vehicle scheduling and driver scheduling are established research areas within public transportation planning. Vehicle scheduling focuses on assigning vehicles to a set of planned trips, while driver scheduling considers the allocation of drivers to the required duties. These decisions are closely related because a trip can only be operated when both a suitable vehicle and an available driver are provided [4], [5].

Boyer et al. [4] examined vehicle and crew scheduling for flexible bus transportation systems and demonstrated the relationship between vehicle allocation and crew requirements. The study shows that scheduling decisions need to consider the characteristics of the transportation service and the constraints associated with available resources.

Andrade-Michel et al. [5] investigated vehicle and reliable driver scheduling in public bus transportation systems. Their work considers the reliability of drivers and the effect of driver availability on scheduling decisions. This is relevant to operational systems because the planned assignment of a driver may no longer be feasible when the driver's availability changes.

These studies provide a foundation for the proposed system's combined consideration of buses and drivers. However, the proposed scheduling component focuses specifically on operational disruption handling after the base schedule has already been generated. When an affected bus becomes unavailable, the system first identifies

feasible replacement buses and subsequently selects an eligible driver for the selected replacement bus.

2.3.2 Dynamic scheduling and real-time operational control

Dynamic scheduling deals with situations where the operational plan needs to change as new information becomes available. Public transport operations may experience changes during the service period, requiring control actions to maintain service continuity. These actions can include rescheduling, vehicle adjustments, and other operational control measures [6], [7].

Gkiotsalitis [6] reviews bus operations optimisation techniques related to bus holding, rescheduling, and stop-skipping. The review demonstrates that operational control can involve modifying existing service decisions in response to changing operating conditions. Gkiotsalitis and Cats [7] similarly discuss control measures that can be applied during public transport operations.

Real-time control research also considers coordination between services and the adjustment of operations as conditions change. Gkiotsalitis et al. [8] review real-time public transport transfer synchronisation and identify different approaches for operational control, including mathematical, heuristic, and rule-based methods.

These studies indicate that operational scheduling should not be considered completely static after the initial schedule has been created. The proposed system follows this principle by maintaining the monthly schedule as the base plan while allowing selected assignments to change when a confirmed disruption occurs.

2.3.3 Disruption management in public transportation

Disruptions can affect public transport services by making previously planned vehicle or driver assignments infeasible. Examples include vehicle failures, staff unavailability, and other operational events. Disruption management therefore involves identifying the impact of an event and determining how the affected service can be maintained using the resources that remain available.

Lai and Leung [9] examined real-time rescheduling and disruption management for public transit. Their work highlights the need to revise operational schedules when

disruptions occur and supports the use of rolling decision horizons for real-time rescheduling.

Zhang et al. [10] examined disruption management in metro systems and substitute bus services, showing that disruption management can involve different response strategies depending on the characteristics and scale of the disruption. Although the operational context differs from the proposed private bus scheduling environment, the work provides relevant evidence that transportation systems require structured responses when normal operations are interrupted.

The proposed system applies disruption management at the level of affected bus trips. Rather than rebuilding the complete monthly schedule after every disruption, the system identifies the affected assignments and evaluates replacement resources. This limits the scope of schedule modification while preserving unaffected assignments.

2.3.4 Disruption scope in rescheduling

The scope of rescheduling is an important consideration in disruption management. A disruption does not necessarily require every assignment in a schedule to be changed. If only one bus becomes unavailable, modifying unrelated trips can introduce unnecessary changes and may create additional scheduling conflicts.

A targeted rescheduling approach therefore focuses on the trips that are directly affected by the operational event. The affected assignments are identified first, after which feasible alternatives are evaluated. This approach is consistent with the operational nature of real-time rescheduling, where decisions are made according to the current state of the system and the impact of the disruption [9].

For the proposed system, disruption scope is defined according to the affected bus and the timing of the disruption. For an approved emergency event, the system evaluates the affected bus's remaining scheduled turns within the relevant operational window. Trips that have already been completed before the disruption are not modified. This allows the system to preserve completed and unaffected assignments while adapting only the remaining part of the schedule.

This approach also supports practical schedule management because a disruption at a particular time does not require the system to reconsider earlier assignments. The resulting schedule therefore reflects both the original plan and the current operational state.

2.3.5 Workload balance in replacement decisions

Replacement vehicle selection is not only a feasibility problem. When several buses are capable of operating the same affected trip, the system also needs a method for selecting one of the feasible candidates. Repeatedly assigning the same vehicle can create an uneven distribution of operational workload.

Vehicle and driver scheduling research demonstrates the importance of considering resource constraints and workforce-related factors when constructing feasible schedules [4], [5]. In an operational replacement scenario, these considerations can be extended by examining the current workload of available buses before making a replacement decision.

The proposed system therefore uses a workload-aware ranking process after infeasible candidates have been removed. Route compatibility is considered first so that the selected bus can operate the required service. Among compatible buses, the system considers the number of trips already assigned to each bus during the current day and the number of recent reallocations. This allows the replacement decision to consider both immediate workload and previous replacement involvement.

This approach does not define fairness as a separate mathematical optimisation objective. Instead, workload is incorporated into the decision rules so that, when multiple feasible buses are available, the system can avoid repeatedly selecting the same bus. A bus identifier can be used as a final deterministic tie-breaker when other ranking criteria produce an equal result.

2.3 Existing systems products and techniques

2.3.1 Optimization-based techniques

Optimization-based scheduling techniques formulate transportation scheduling as a mathematical decision problem. Variables can represent vehicle assignments, driver assignments, departure decisions, or other operational choices, while constraints define the conditions that must be satisfied. An objective function can then be used to minimize cost, reduce operational inefficiency, or improve another defined performance measure [1], [4].

These techniques are useful when the scheduling problem is sufficiently defined and the required objectives can be represented mathematically. Vehicle and crew scheduling research has demonstrated the application of optimization methods to transportation resource allocation [4], [5].

However, operational disruption handling can introduce additional complexity because the scheduling decision must be made using the current state of vehicles and drivers. A complete optimization process may also require repeated recalculation when individual disruptions occur. For the proposed system, the scheduling component therefore uses explicit operational rules for the disruption response instead of attempting to optimize the entire monthly schedule after every event.

2.3.2 Real-time control techniques

Real-time control techniques are used to modify public transport operations as the service is being delivered. These techniques may respond to changing operating conditions and can include measures such as rescheduling, holding, short-turning, stop-skipping, and service coordination [6]–[8].

The main characteristic of real-time control is that decisions are based on operational information that becomes available during service execution. This makes real-time control relevant to disruption handling because a schedule prepared before the operating day may not remain fully feasible once an unexpected event occurs.

The proposed system adopts the same general principle of responding to current operational conditions but applies it to a more specific replacement scheduling problem. When a confirmed disruption occurs, the system evaluates the affected trips and the current state of buses and drivers. It then updates only the affected part of the schedule where a feasible replacement is available.

2.3.3 Rule-based techniques

Rule-based scheduling uses predefined conditions to guide decisions. Instead of searching for a mathematically optimal solution, the system evaluates a sequence of operational rules and selects an action that satisfies the required conditions.

For example, a replacement bus can first be checked for availability. If the bus has a conflicting assignment, it can be removed from the candidate set. The system can then check turnaround requirements and route compatibility. After feasible candidates have been identified, workload-related rules can be applied to determine the preferred replacement.

Rule-based techniques are particularly suitable when operational decisions need to be transparent and easy to explain. They can also provide predictable responses to recurring operational events. Research on public transport real-time control identifies rule-based approaches alongside mathematical and heuristic approaches as possible methods for operational decision-making [8].

The proposed system follows a rule-based workflow for disruption handling. The rules are applied in a defined sequence: identify affected trips, filter infeasible buses, rank feasible buses according to workload-related criteria, select a suitable driver, and either reallocate the trip or cancel it when no feasible combination exists.

2.3.4 Comparison of techniques

The reviewed techniques address different aspects of public transport scheduling. Optimisation-based approaches are suitable when a clearly defined objective needs to be optimised across a set of constraints. Real-time control techniques focus on modifying operations as conditions change. Rule-based approaches provide explicit

and traceable decision logic, while data standards support the representation and exchange of updated transportation information.

For the proposed scheduling component, these techniques are not treated as mutually exclusive concepts. The system adopts the practical aspects that are relevant to its operational requirements. The monthly base schedule follows a timetable-driven scheduling process, while disruption handling uses rule-based feasibility checks and workload-aware ranking. Operational state information provides the data required for these decisions, and standardized real-time formats can be used when schedule updates need to be communicated externally.

The main distinction is therefore in the role of each technique. Optimization focuses on selecting decisions according to a mathematical objective, real-time control focuses on operational adjustment, rule-based methods focus on explicit decision conditions, and data standards focus on information representation and exchange. The proposed system primarily uses rule-driven operational adaptation because the research focuses on explainable and targeted rescheduling after specific disruptions.

2.4 Gap synthesis and conceptual framework

2.4.1 Synthesis of the literature

The literature reviewed in this chapter shows that public transport scheduling is a combination of timetable planning, vehicle scheduling, driver scheduling, and operational control. The scheduling process needs to consider both the planned service requirements and the resources available to operate those services [1], [2].

Existing research provides several approaches for solving these problems. Optimisation methods can model complex scheduling decisions using objectives and constraints, while heuristic and rule-based methods can provide practical approaches for making operational decisions [1], [6], [8]. Research on vehicle and driver scheduling further demonstrates the importance of considering resource availability and workforce constraints when assigning services [4], [5].

The literature also indicates that the initial schedule cannot always be treated as fixed throughout the operating period. Operational disruptions may require rescheduling and

adjustment of affected services [6], [9]. Disruption management research shows that transportation systems need structured methods for responding to unexpected events and maintaining service continuity where possible [9], [10].

Based on these findings, the proposed system considers bus scheduling as a continuous process rather than a one-time schedule generation task. A monthly base schedule provides the planned service structure, while operational state information is used to support decisions during daily operations. When a disruption occurs, only the affected part of the schedule is reassessed, and replacement resources are selected using predefined feasibility and workload-related rules.

Literature area	**Existing focus and strength**	**Limitation for the proposed context**	**Proposed contribution**
Vehicle scheduling	Assigns vehicles to scheduled trips [1], [2]	A planned assignment can become infeasible after a breakdown	A planned assignment can become infeasible after a breakdown
Crew/driver scheduling	Assigns drivers under duty and labor constraints [4], [5]	Driver selection is often part of a larger optimization problem	Select the driver after the replacement bus, by availability and daily workload
Integrated vehicle-driver scheduling	Joint optimization with globally feasible assignments [4], [5]	Computational complexity grows with constraints and fleet size	Use a lightweight sequential process for operational response
Real-time control	Holding, dispatching, rescheduling [6]–[8]	Often centered on individual control actions	Provide an event-driven sequence from disruption

			event to updated schedule
Disruption management	Recovery after unexpected events [9], [10]	May use complex optimization or specific contexts such as metro [10]	Apply explicit operational rules for breakdown and approved-leave cases

Table 1: Gap Analysis of Existing Scheduling Approaches against the Proposed Component

2.4.2 Identified research gap

The gaps identified in the literature reviewed are threefold.

First, planning and recovery from disruption have been treated separately in the literature. Some literature offers good approaches to timetable construction and vehicle scheduling, while other literature analyzes real-time disruption recovery. However, for a practical implementation of the system, continuous interaction between the original schedule and the operational condition after the disruption is a must.

Second, choosing replacement resources is not always a clear operational decision-making process. Mathematics and metaheuristics can provide proper schedules, but the increase in computational complexity is likely. For the operation of the bus management system, the decisions should be made in a comprehensible manner. In the proposed algorithm, eligibility filtering, ranking of buses, choosing drivers, and making reallocation decisions are distinct.

Third, fairness in redistribution in repeated disruptions requires explicit consideration. Simply choosing any bus available may result in repeated utilization of the same resource. The suggested approach incorporates the daily count of trips and the frequency of redistribution in the ranking process, thereby taking into account the operational load along with the suitability of the route.

Thus, the research gap identified in this research work can be expressed as follows:

There is a need for a practical, event-driven bus scheduling workflow that connects the monthly base schedule with operational time-slot management and automatically

handles disruptions by identifying affected trips, filtering infeasible replacement buses, fairly ranking eligible buses, selecting an appropriate driver, and updating or cancelling the affected schedule while maintaining operational constraints.

This gap directly supports the proposed Disruption-Aware Dynamic Schedule Adaptation Algorithm and its integration within the broader Policy-Driven Bus Scheduling and Time-Slot Management Engine.

2.4.3 Conceptual framework

The conceptual framework of the proposed scheduling component connects planned scheduling with operational adaptation. The process begins with the timetable, which defines the required trips, directions, and operating times. These trips form the basis for generating the monthly base schedule.

The next stage considers the available buses and drivers. The system assigns buses to timetable trips while checking route compatibility, existing assignments, and turnaround requirements. Drivers are then associated with the required operations according to their availability and workload conditions.

During daily operations, the system maintains the current operational state. When a disruption event is confirmed, the event becomes the trigger for schedule adaptation. The system identifies the affected trips within the relevant operational period and generates a set of candidate replacement buses.

The candidate buses are filtered using operational constraints. Buses that are unavailable, already assigned to conflicting trips, incompatible with the required route, or unable to satisfy the turnaround requirement are excluded. The remaining candidates are ranked using workload-aware criteria. Route compatibility is considered first, followed by the number of trips assigned during the current day and recent reallocation involvement. A deterministic bus identifier can be used when a final tie-breaker is required.

After a replacement bus is selected, the system searches for an eligible driver. The regular driver associated with the selected bus is preferred when available. If that driver cannot operate the trip, another eligible driver can be selected based on

availability and workload. If both a feasible bus and driver are found, the affected assignment is updated. If no feasible combination exists, the trip is cancelled and the reason is recorded.

The resulting conceptual workflow can be represented as:

1. **Monthly base schedule:** the planned allocation of trips and buses for the month.
2. **Time-slot / operational state:** the current state of each bus and driver, changing with completed trips, availability, leave, and disruption events.
3. **Disruption event:** a confirmed event, such as a bus becoming unavailable or approved leave, starts the process.
4. **Affected trips:** trips that have become infeasible are identified within the disruption window: a rolling 24-hour window for an approved emergency, or the stated date or time range for a normal leave or operational request.
5. **Candidate bus filtering:** buses that cannot take a trip because of unavailability, schedule conflict, or turnaround constraints are removed.
6. **Fair bus ranking:** the remaining candidates are ranked by route compatibility, then by trips already assigned that day, then by recent reallocations.
7. **Reallocation or cancellation:** the trip is reallocated to the selected bus and driver. If no feasible combination exists, it is cancelled, and the reason is recorded.
8. **Updated schedule:** the revised allocation replaces the affected part of the operational schedule, and unaffected assignments are preserved.

This framework provides a clear relationship between the planned schedule and the operational response. It also ensures that disruption handling does not automatically require the complete schedule to be regenerated.

3 RESEARCH METHODOLOGY

3.2 Research design

The research utilizes a Design Science Research (DSR) methodology since the output of the research will be a technological device designed to solve a certain operational problem. The device used in this case is a Policy-Driven Bus Scheduling and Time-Slot Management system, while the key method is the Disruption-Aware Dynamic Schedule Adaptation Algorithm. The problem at hand relates to difficulties in the maintenance of an established schedule within a month when a bus is suddenly cancelled.

Based on this fact, the principle of rule-based design was preferred to optimization-based design since the operating specifications can be presented as constraints and priorities, while each action should be based on an explicit rule. Research in real-time rescheduling illustrates that schedules can be revised in real time due to changes in operational data [1], while the review of bus operations indicates that rescheduling is an important way of responding to a service disruption [2]. This research assumes this principle but puts it into practice as a defined process of decision-making.

Stage	**Activity**	**Outcome**
1	Problem identification	Definition of the schedule continuity problem under bus unavailability
2	Review of existing approaches	Identification of the research gap (Chapter 2)
3	Definition of objectives and constraints	Preparation time, availability, assignment conflict, and turnaround constraints
4	Design of the artefact	Monthly base scheduling mechanism, time-slot

		management model and adaptation algorithm
5	Implementation	Working component within the IoT-enabled smart bus system
6	Scenario-based evaluation	Comparison of expected and actual system decisions
7	Analysis	Findings, trade-offs and limitations

Table 2: Stages of the Research Workflow

Evaluation is carried out through controlled scenarios, because the correctness of each decision can then be checked against rules defined before the experiment.

3.3 Data participants or study materials

The element functions using organized operational data. The method doesn't rely on a machine learning model for decisions and doesn't require data associated with passengers. Consequently, the data used for the evaluation consists of operational data that is simulated and has a character compatible with the schema of the production database and is representative of buses, drivers, routes, trips and assignments.

The main data entities are listed below.

Entity	**Required Information**
Bus	Bus ID, route, availability, duty status
Driver	Driver ID, regular bus, availability, daily workload
Route	Route ID and route information
Trip	Trip ID, route, date, departure time
Assignment	Trip, bus, driver, status
Disruption	Event type, affected bus, event time

Reallocation	Original bus, replacement bus, reason, timestamp

Table 3: Operational Data Entities and Required Information

The record for each task contains data that is essential for calculating the workload of each bus and driver, as well as the relevant recent reallocations. Those are the two numbers on which the ranking phase relies.

The scenario datasets are created in such a manner that each scenario represents only one decision-making rule. For instance, a scenario in a turnaround constraint consists of a potential bus whose last assignment finished extremely close to the corresponding trip departure while remaining valid in all the other aspects.

3.4 Experimental or study procedure

Assessment is made in three stages in accordance with the operational life cycle of the scheduling component.

First stage: Monthly base planning. The system generates a monthly base schedule through the allocation of scheduled trips to vehicles. The allocation process gives preference to a suitable vehicle having the least number of currently assigned trips. The outcome is verified against two criteria: completeness (all scheduled trips should be assigned to a vehicle) and absence of conflicts (no vehicle can have overlapping assignments).

Step 2: Time-slot management based on disruption. In case of disruption of a bus, the system tracks time slots for the bus that is not in operation. For all such time slots, the system obtains information relative to the trip which the bus was supposed to make, including date, route, departure time, bus assigned, driver assigned, and the status of the trip. The impacted slots are then either reassigned or canceled in accordance with the relevant rules. The slots that are successfully reallocated contain new information about the bus and the driver. However, for the slots for which there is no possibility to take measures, they become CANCELLED along with the reason. The original record

for the relevant schedule remains unchanged; therefore, the system is able to trace the slots that have changed after the disruption occurred.

Step 3: Rescheduling with the account for disruption. The system recognizes an instance of disruption that occurred in connection with the scheduled bus. After that, the management of the relevant slots is undertaken in such a way:

1. Determine the reallocation window (rolling 24 hours for an emergency, or the requested date or time range for a specific-date operation);
2. Retrieve the affected trips of the unavailable bus within that window;
3. Check the preparation time of each affected trip;
4. Perform candidate bus filtering using availability, duty status, assignment-conflict, and turnaround constraints;
5. Rank the remaining candidates by route compatibility, daily workload, and recent reallocations;
6. Select the highest-ranked bus;
7. Perform driver selection, using the regular driver of the selected bus when available and the available driver with the lowest daily workload otherwise;
8. Apply the reallocation and update the affected time slot with the selected bus and driver, then notify the relevant stakeholders;
9. Apply cancellation and record the reason when preparation time, bus or driver requirements cannot be met.

All impacted slots are treated separately, so the result of one time slot does not affect the decisions of any other time slot. Nonetheless, prior reallocation produces an operational state that has an effect on the selection of candidates used for the next time slots, like the level of work needed for the day or the availability of the replacement bus.

The scenarios are prepared in advance and can be found in Appendix C (T01 to T15). They consider normal operations without disruption as well as types of reallocation windows, checks of constraints, various criteria of ranking, and points of driver

selection and cancellations. Starting from the same freely produced monthly schedule for every scenario, the outcomes will be comparable.

3.5 Measures and evaluation metrics

In evaluation measures, operational scheduling is preferred over classification metrics since it focuses on rule-based scheduling and reallocation decisions instead of predictive classifications.

The Reallocation Success Rate is defined as the ratio between the number of successfully reallocated trips to the total number of affected trips. The greater the percentage, the more likely the system successfully fulfills scheduled trips during a disruption.

Cancellation Rate is defined as the ratio of the number of trips that could not be executed, hence cancelled. A trip is considered cancelled when a feasible bus and driver are unavailable, or when the preparation time remaining is not enough. Cancellation Rate is calculated as the ratio of the number of trips cancelled to the number of trips affected. Since every affected trip is either reallocated or cancelled, the two ratios represent all affected trips in a single test run.

The Constraint Compliance Rate shows how well reallocation actions meet the pre-made operational requirements. A decision is said to be compliant if the chosen reallocation does not have any scheduling conflict, meets the required turnaround time, employs an available bus and driver, and adheres to the rules of candidate selection. The rate is calculated by comparing the number of compliant decisions against all the decisions under review.

The Average Reallocation Processing Time shows the amount of time spent by the system to process an affected trip, from the moment a reallocation is made until the trip is canceled or reallocated. The value is calculated through the sum of processing times of affected trips divided by the total number of trips being processed.

The evaluation of Workload Distribution checks the situation of trips allocated to different eligible vehicles during allocation process before and after handling disruptions. The original and reallocated trips are checked so that the following calculations of this ratio according to standard deviation can be done. Its smaller value shows that the process of distribution of excess work is more even.

3.6 Data analysis

Analysis is performed for each scenario and then for more than one scenario. The expected result for each scenario is set prior to execution and compared to the actual system outcome. A scenario is considered to be successful if the trips affected by the decision, the eliminated candidates, which buses were selected, the driver chosen for the trip, and the final outcome match the expected one.

The following items are recorded for every scenario:

affected trips, the eliminated candidates; chosen bus, ranking of the latter; chosen driver; cancellation or reallocation results, and the stored reason; processing time. The results of each scenario are analyzed with the help of established operational parameters. The Reallocation Success Rate, Cancellation Rate, and Constraint Compliance Rate are obtained from the outcomes of the scenarios. The Average Processing Time of Reallocation is calculated on the basis of the average processing time and the best and worst processing times seen in various runs of the scenario. This helps eliminate the problems with one-off deviations of the execution conditions.

3.7 Ethics privacy security and responsible AI

The scheduling function does not require information about passengers to function. The information concerning the bus and drivers is treated as operational and is provided only following the privacy and access control regulations of the entire system. Driver workload information is only used as an operational parameter for trips allocation and is not used for assessing their work performance.

The access to scheduling and reallocation functions is limited to approved users via role-based access control. Schedule alterations are recorded with all the relevant

information, including the original bus, a substitute bus, the reason for the change, and the timestamp. This guarantees the traceability of operations and auditing.

The function is implemented according to the specified rules and does not use generative or predictive AI for scheduling or reallocation purposes. Therefore, the major considerations in relation to the responsible system are transparency, explanation, and accountability. There is a possibility to trace back every cancellation/reallocation by the pre-defined constraints, rules of selection of candidates, and criteria of decision-making.

Disclosure of the use of generative AI and other robots used in the R&D process is in accordance to the thesis guidelines. The author is responsible for confirming truthfulness, originality, citations, data, and outputs included in the research.

3.8 Reproducibility and research artefacts

Reproducibility is ensured because artefacts are kept, allowing each test case to be repeated in the same conditions and with the same system configurations. The artefacts include the following:

- The project management tool for managing the scheduling and disruption handling components;
- The database schema and data with which the monthly base schedule was created;
- The datasets containing the scenarios and scenario matrix specified in Appendix C;
- Configuration values—including those related to minimum preparation time, turnaround periods, reallocation windows and candidate ranking;
- Algorithm pseudocode provided in Appendix A;
- Testing scripts and API testing collections used during testing;
- Decision logs with candidate buses, reasons for rejecting those buses, selected resources, and results;
- Screenshots show how the schedule changed before and after handling the disruptions.

The reallocation method follows fixed rules and selection considerations. Thus, the same seed data, operating conditions, and configuration would lead to the same schedule. The elapsed time may vary between executions because of different execution environments. Thus, because elapsed time is collected from repeated executions, the use of summary statistics instead of data from one execution is necessary.

These documents contain the necessary information to follow the input conditions, the decision-making process, and the result of a particular experimental situation.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements define the main functions and quality expectations of the Policy-Driven Bus Scheduling and Time-Slot Management component. They are divided into functional and non-functional requirements.

4.1.1 Functional Requirements

ID	Requirement	Description
FR1	Monthly base scheduling	The component shall produce a monthly base schedule by allocating scheduled trips to available buses.
FR2	Time-slot management	The component shall keep every scheduled turn together with its route, date, departure time, assigned bus, assigned driver, and status.
FR3	Disruption registration	The component shall accept a disruption event that makes an assigned bus unavailable.
FR4	Affected trip identification	The component shall retrieve the future turns of the affected bus that fall inside the applicable reallocation window.
FR5	Preparation-time validation	The component shall cancel an affected trip when the time left before departure is shorter than the minimum preparation time.
FR6	Candidate bus filtering and ranking	The component shall remove ineligible buses and rank the remaining candidates by route compatibility, daily workload, and recent reallocations.

FR7	Reallocation	The component shall write the selected bus and driver into the affected turn and mark it as reallocated.
FR8	Cancellation	The component shall cancel an affected trip and store the reason when no feasible bus or driver exists.
FR9	Notification	The component shall inform the relevant stakeholders after every successful reallocation.

Table 4: Functional Requirements of the Scheduling Component

FR1: Monthly Base Scheduling

A monthly schedule must be generated prior to the commencement of operations. Then trips are assigned to the existing fleet based on operational needs of the route.

For every trip scheduled, the buses able to service a particular trip are determined, and the bus with the lowest number of trips assigned is chosen. The resulting schedule is termed 'planned operational status'.

Success requirements:

- Every planned trip must be assigned a bus, if at least one eligible bus exists
- No bus can be assigned to two overlapping trips
- Assignments must be proportionally split among the bus fleet.

FR2: Time-Slot Management

The component is expected to interpret the generated schedule as the set of operational trips, not the rigid timetable. Each trip saves its route, date, departure time, assigned bus, assigned driver, and status.

A trip status may be changed through the daily operations, for example, from SCHEDULED to REALLOCATED or CANCELLED. The initial record of the trip is staying along with the status change.

Success criteria:

- Each trip is retrievable by the bus, route, date, and period of time.
- Status change does not erase the trip assignment information.
- Trips are evaluable independently during the daily operations.

FR3: Disruption Registration

The component must be capable of registering any disruption incident which renders a bus that is part of a scheduled trip unavailable. Such incident may be caused by the breakdown of the bus, receipt of an official message about unavailability of the bus, any details of a missed journey, or another operational disturbance. The details of the incident include the actual bus, its category, and the time of its occurrence. In case of an emergency incident, the reallocation timeframe applies for 24 hours from the moment of incident. In case of a specific date journey, the timeframe needs to be specified at the time of booking.

Success criteria:

- The disruption incident can be recorded for any scheduled bus.
- The correct reallocation timeframe is specified for every category of incident.
- Emergency incidents extend across midnight if needed.

FR4: Affected Trip Identification

The subsystem must capture the future trips of the affected vehicle that exist in the period of reallocation. No switches outside this time period nor any other affected vehicle will be changed. Each affected trip is to be treated separately. Thus, the cancellation of one trip will not affect the outcome of another trip.

Success criteria:

- All and only the future trips of the affected vehicle will be selected.
- Trips in the specified date range are the only ones that are being processed.

FR5: Preparation-Time Validation

Before looking for a replacement, the component will verify how much time is left before departure. If the time left is less than the minimum preparation time, then the trip is cancelled and the reason will be noted.

This check ensures that the component does not find a bus and driver who cannot reach the departure location in time.

Success criteria :

- A trip that has insufficient preparation time will be cancelled in an orderly manner without searching for a replacement.
- The reason for the cancellation cites insufficient preparation time as a reason.

FR6: Candidate Bus Filtering and Ranking

First, the system will eliminate any buses that are not able to perform the scheduled trip. The eliminated buses can be unavailable, be already performing some other trip at the time it has to serve the affected trip, and have too little time for recovery between trips.

The remaining buses will be prioritized regarding three criteria: compatibility with the trip, volume of work on the day, and the number of recent reallocations. First, the buses that are operating in the affected route will be prioritized, followed by those that have less workload for the day and have not been used recently.

Success Criteria:

- All buses that are not eligible do not reach ranking.
- The ranking is carried out according to the prioritized criteria in all test cases.
- The reasons for rejecting a particular bus will always be explained.

FR7: Reallocation

The module shall allocate the bus with the highest rank to the trip that is affected and select a driver accordingly. The bus driver who regularly drives the bus is considered first. If the driver is unavailable, then a driver who has worked least that day is chosen.

When both a bus and a driver are identified, the turn is updated with the new bus and assigned driver, updating its status to REALLOCATED.

Success Criteria:

- A driver will be selected only after selecting a replacement bus.

- The preferred bus driver will be chosen when available.
- The updated turn consists of only one bus and one driver.

FR8: Cancellation

Whenever there are no viable alternatives to replace a journey, the system has to cancel that trip. This can occur in cases of insufficient prep time, lack of a proper bus, or lack of a qualified driver for the bus chosen.

The status changes to CANCELLED, and the cancellation reason is attached to an event.

Success criteria:

- No trip should remain without assignment or decision
- Every cancellation has the reason recorded.

FR9: Notification

Every time there's a reallocation, the system will alert the legitimate stakeholders concerned. Notifications will provide information about the primary bus, substitute bus, driver chosen, the turn affected, and the scheduling data.

Success criteria:

- A notification is sent for every rearrangement,
- Content of notifications matches the records of the revised schedule.

4.1.2 Non-Functional Requirements

NFR1: Explainability

Each step executed by the system is in accordance with a precise regulation. A bus gets a refusal due to a specific restriction applied, and a bus gets a preference based on its score in the previously established hierarchy. In this way, stewards and operators will always have an explanation for the change made rather than accepting results without rationale.

NFR2: Traceability

All details about the original task, the event that triggered the change, the decision, and motivation are saved. The reason for the schedule change is known after the change has been implemented.

NFR3: Consistency

Schedule updates are executed in full. A turn after an update has one bus and one driver, and neither is simultaneously designated for a conflicting assignment.

NFR4: Responsiveness

The decision for every affected journey must be made in a timely manner in order for stakeholders to take necessary actions during business operations. The duration of actual processing is calculated within the frameworks of case studies.

NFR5: Configurability

Policy parameters like minimum preparation time and turnaround requirement remain configuration parameters and can be modified by the transport authority without altering the algorithm architecture.

NFR6: Access Control

Only those who have approval can access the functions for creating or changing a schedule. Other users can only check the schedule as it relates to their work. Authorization and role-based permissions are used for this separation of functions.

4.2 Component architecture within the complete solution

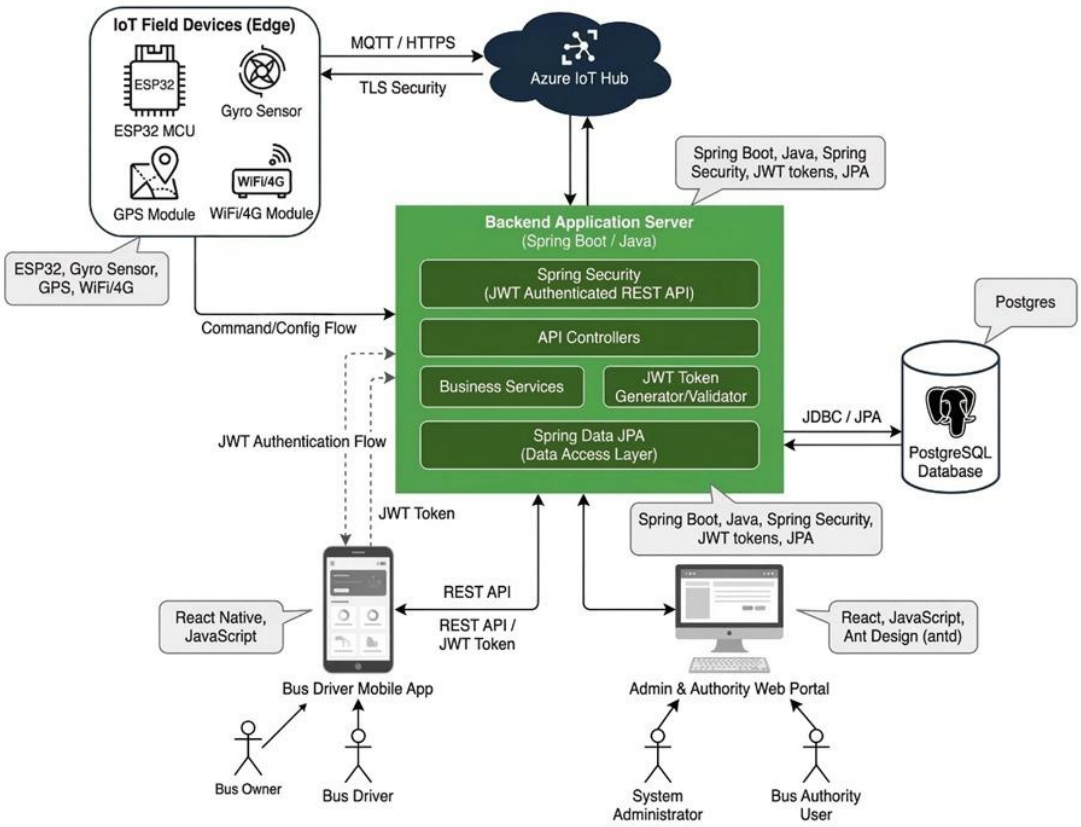


Figure 1: Overall System Architecture of the IoT-Enabled Smart Bus Solution

The scheduling component is part of the wider IoT-Enabled Smart Bus Route Monitoring and Revenue Management System. The complete system contains the following major components:

- **IoT-based vehicle monitoring and route compliance:** Collects vehicle telemetry and monitors route progress.
- **Policy-driven scheduling and time-slot management:** Generates the monthly base schedule and adapts it when disruption events occur. This is the component addressed in this thesis.
- **Distance-based revenue management:** Calculates revenue from verified operations.
- **Driver's behavior monitoring:** Observes driver behavior and status.

- **Shared platform:** Spring Boot backend, PostgreSQL database, and Azure IoT Hub, together with the Authority, Owner, and Driver portals.

4.2.1 Scheduling Component Architecture

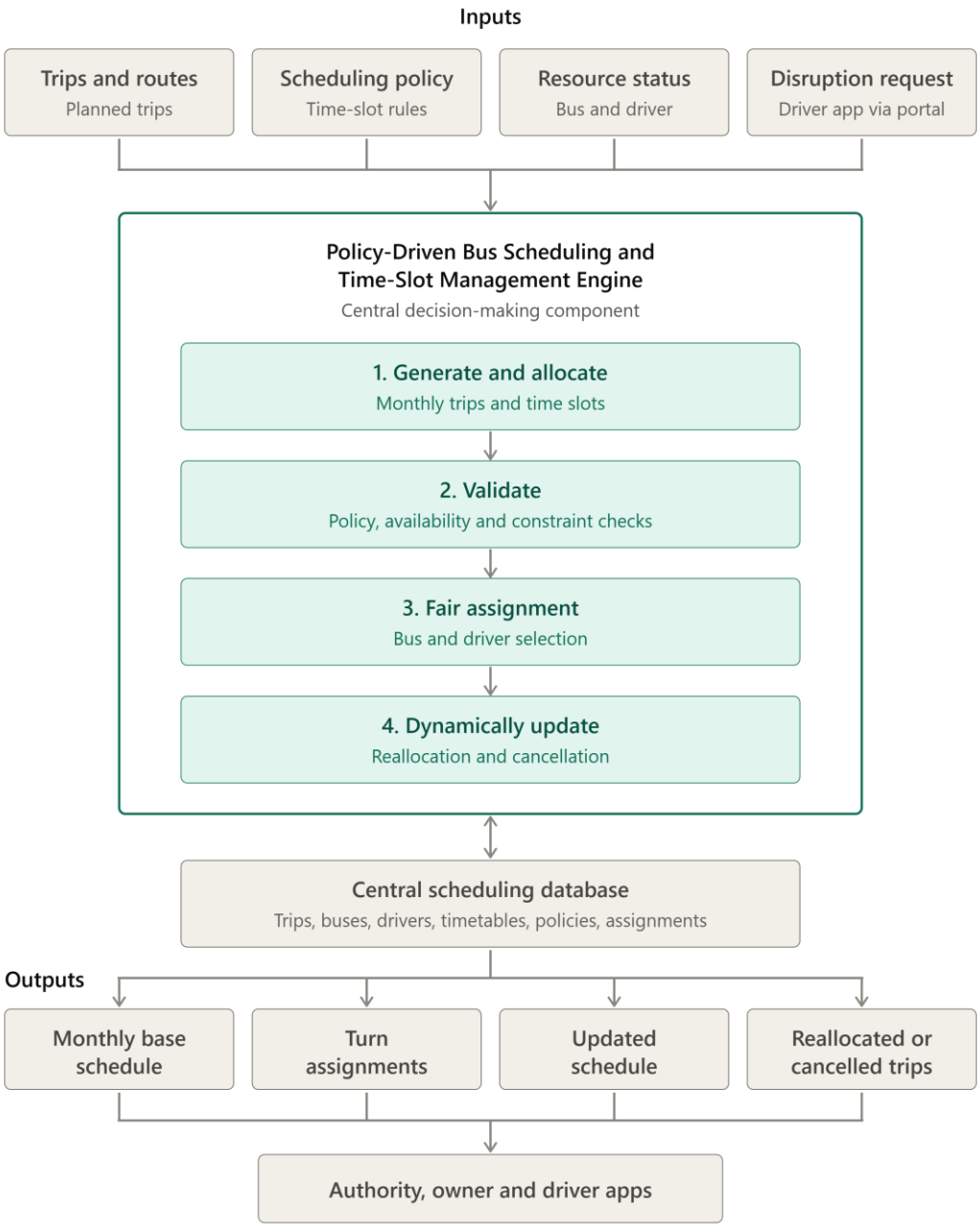


Figure 2: Architecture of the Policy-Driven Bus Scheduling and Time-Slot Management Engine

The scheduling component contains the following major modules:

Module	**Role**
Operational inputs	Planned trips, bus availability, driver availability, route information, and disruption events
Monthly scheduling engine	Builds the monthly base schedule
Turn repository	Stores every scheduled turn and its status
Time-slot management	Exposes turns for evaluation during daily operation
Disruption handler	Receives a disruption event and fixes the reallocation window
Schedule adaptation engine	Runs the Disruption-Aware Dynamic Schedule Adaptation Algorithm
Schedule database	Stores reallocations, cancellations, and reasons
User interfaces	Present the updated schedule to authorities, owners, and drivers

Table 5: Major Modules of the Scheduling Component

The backend layer performs scheduling, ranking, and reallocation. The database stores turns, assignments, disruption events, reallocation records, and cancellation reasons. The adaptation engine never builds a new monthly schedule. It changes only the turns that a disruption event has put at risk.

4.3 Design alternatives and decisions

Four approaches to replacing an unavailable bus were considered.

Alternative	Description
Manual reallocation	A timekeeper searches for a replacement bus and driver
First available bus	The first bus marked available is assigned
Optimization-based rescheduling	A mathematical model searches for a globally best reassignment, as in rolling-horizon vehicle and crew rescheduling.
Rule-based fair reallocation	Ineligible buses are removed and the rest are ranked by fixed priorities

Table 6: Design Alternatives for Replacing an Unavailable Bus

4.1 Detailed design

4.1.1 Service Design

The component is built as a set of services inside the Spring Boot backend.

The main services are:

- **Monthly Scheduling Service:** Generates the monthly base schedule.
- **Turn Management Service:** Stores and retrieves operational turns.
- **Disruption Handling Service:** Registers disruption events and fixes the reallocation window.
- **Schedule Adaptation Service:** Runs the Disruption-Aware Dynamic Schedule Adaptation Algorithm.
- **Bus Ranking Service:** Applies candidate bus filtering and ranking.
- **Driver Selection Service:** Selects the regular driver or an alternative available driver.

- **Notification Service:** Informs stakeholders about successful reallocations.

4.1.2 Monthly Base Scheduling

The monthly base schedule represents the planned operational state. For each scheduled trip, the engine selects the eligible bus with the smallest number of assigned trips:

```
SelectedBus = the eligible bus b with the minimum AssignedTrips(b)
```

The collected assignments form the monthly base schedule.

4.1.3 Operational Turn Representation

Each scheduled turn is stored as a record:

```
Turn = (Date, Route, DepartureTime, Bus, Driver, Status)
```

The status moves from SCHEDULED to REALLOCATED when a replacement is found, and from SCHEDULED to CANCELLED when none exists. Because only the status and the assigned resources change, the planned record stays available for comparison with the updated schedule.

4.1.4 Disruption Handling and Reallocation Window

A disruption event is triggered by the unavailability of the scheduled bus. The bus and its regular driver are not treated as one inseparable resource at any later stage.

For an emergency, the window ends 24 hours after the event time:

```
Window_end = CurrentTime + 24 h
```

For a specific-date operation, the window is the requested range:

```
Window = [StartDateTime, EndDateTime]
```

4.1.5 Affected Trip Identification and Preparation-Time Check

A turn T is an affected trip when it is assigned to the unavailable bus and lies inside the window:

```
Affected(T) = (AssignedBus(T) = B_affected) AND (T in Window)
```

Before any search begins, the time remaining before departure is compared with the minimum preparation time:

```
If PreparationTime(T) < P_min, then Status(T) = CANCELLED
```

4.1.6 Candidate Bus Filtering

A candidate bus B is kept only when all of the following conditions hold:

```
Eligible(B, T) = Available(B) AND NOT OffDuty(B) AND NOT
Conflict(B, T) AND TurnaroundOK(B, T)
```

Buses that are unavailable, off duty, already assigned at the same time, or without sufficient turnaround time are removed before ranking begins.

4.1.7 Bus Ranking

The remaining candidates are ordered by route compatibility, then daily workload, then recent reallocations:

```
Priority(B) = (RouteMatch(B), -DailyTrips(B), -
RecentReallocations(B))
```

The last two criteria act as tie-breakers. The recent reallocation criterion prevents the same bus from absorbing every disruption simply because its other values are favorable.

4.1.8 Reallocation and Cancellation Decision

When an affected trip is identified, the system first verifies whether there is sufficient preparation time available. If the preparation time is insufficient, the trip is cancelled, and the specific reason is recorded in the system. If sufficient preparation time exists, the system then searches for an eligible replacement bus. If no suitable bus is available, the trip is cancelled and the reason is stored; otherwise, the system proceeds to check for an available driver.

If no driver can be found, the trip is likewise cancelled with the cancellation reason saved. However, if a driver is available, the trip turn is updated with the newly assigned bus and driver, and its status is changed to "REALLOCATED." Finally, all relevant stakeholders are notified. In all cases, cancellation serves as a controlled fallback mechanism to ensure that the updated schedule contains no invalid or unfulfillable assignments.

4.1.9 Notification

After a successful reallocation, stakeholders receive the original bus, the replacement bus, the selected driver, the affected turn, and the updated schedule information. This links each disruption event to the change it caused.

4.1.10 User Interfaces

The React portals present the schedule to three user groups:

- **Authority portal:** The monthly schedule, the affected bus, the affected trips, the replacement bus, the selected driver, cancellation reasons, and the status of each turn.
- **Owner portal:** The schedule of the owner's buses, including turns moved to or away from them.
- **Driver portal:** The driver's assigned turns and the alert generated when a turn is reallocated to that driver.

4.2 Standards and professional practice

The scheduling component follows common software engineering, security, database, and testing practices.

4.2.1 Software Development Standards

The backend is written in Java and organized into separate services with clear responsibilities. Git and GitHub are used for version control. Testing includes:

- unit testing of decision rules using JUnit and Mockito;
- integration testing of API endpoints and data flow using Postman;
- end-to-end testing from a disruption event to the updated schedule.

4.2.2 Security Standards

Security is considered across the component. The main practices are:

- TLS for data transmission;
- JWT authentication for API requests;
- role-based access to schedule-changing functions;
- input validation on disruption requests;
- database access through Spring Data JPA, which avoids hand-built query strings.

4.2.3 API and Database Standards

The backend follows REST principles, with suitable HTTP methods and status codes for schedule generation, turn retrieval, disruption registration, and result retrieval. PostgreSQL stores buses, drivers, routes, trips, assignments, disruption events, and

reallocation records. Schedule updates are applied inside transactions so that a reallocation is either fully stored or not stored at all.

4.2.4 Logging and Traceability

Every decision is logged with its reason. This supports the traceability requirement and allows an authorized user to review how a schedule change was made.

4.2.5 Documentation and Maintenance

The database schema, algorithm pseudocode, configuration values, and test scenarios are documented so that the results can be reproduced under the same input conditions. Policy values are kept as configuration, which allows the component to be adapted to different transport policies without changing the structure of the algorithm.

4.2.6 Responsible System Practice

The scheduling mechanism is rule-based rather than generative or predictive, so the main responsible-system concern is transparent and auditable decision-making. Driver and operational data are handled according to the privacy and security requirements of the wider project, and access to scheduling functions is restricted to authorized users.

Overall, the scheduling component is designed to be explainable, modular, and able to operate on its own while integrating with the wider IoT-enabled smart bus framework.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The component was developed with the same technologies as the wider project, so it shares the backend an**d the database with the other components.**

Layer	**Technology**	**Purpose in the component**
Programming language	Java	Scheduling, ranking, bus selection, and reallocation logic
Backend framework	Spring Boot	Service layer and REST endpoints
Database	PostgreSQL	Storage of turns, assignments, disruption events, and reallocation records.
Data access	Spring Data JPA	Mapping of scheduling entities to database tables
Authority web portal	React	Schedule display and disruption request handling.
Security	TLS, JWT	Protected transport and authenticated API requests
API testing	Postman	Verification of endpoints and stored results
Development environment	IntelliJIDEA	Backend development
Version	Git / GitHub	Source management

Table 7: Technology Stack Used in the Component

The IoT layer of the wider project uses ESP32 devices and Azure IoT Hub for vehicle telemetry. The scheduling component does not depend on that layer for its decisions, as Section 5.3 explains.

5.2 Component implementation

5.2.1 Monthly Scheduling Service

The monthly scheduling service obtains the trips that need to be scheduled, the buses that are available, and the requirements of the routes for the month. For each scheduled trip, the system examines the buses that can perform it and chooses the one that has the least trips already assigned. The bus that is selected is kept together with the trip, and this way, a monthly plan of assignments is made.

By selecting the bus that has the least number of scheduled trips beforehand, it is possible to create an efficient redistribution of loads in advance and ensure that the reallocation stage goes easier.

5.2.2 Turn Management Service

The turn management function handles the individual service assignments that make up the operational schedule. A turn represents one planned bus operation for a specific date, route, and departure time.

The scheduling services use these turn records when retrieving the existing schedule and when identifying assignments affected by a disruption. Each turn maintains the information required to determine its current operational state and assigned resources.

The main information associated with a turn includes:

- operating date;
- route;
- departure time;
- assigned bus;
- assigned driver;

- schedule status; and
- reallocation or cancellation information, where applicable.

Using individual turns as the main scheduling unit allows the system to modify only the affected assignments. Therefore, an approved disruption does not require the complete monthly schedule to be generated again.

5.2.3 Disruption Handling Service

The disruption handling function manages disruption requests submitted by drivers through the Driver App. The request does not directly change the schedule when it is submitted.

After the request is received, it is made available to the authorised user through the Authority Web App. The authority user reviews the reported disruption and decides whether it should be approved or rejected.

If the request is rejected, the existing schedule remains unchanged. If it is approved, the scheduling component identifies the affected bus and determines the applicable period for schedule adaptation.

The system then retrieves the future turns assigned to the affected bus within the selected period. These turns become the input for the dynamic rescheduling process.

This implementation separates **disruption reporting** from **schedule modification**, ensuring that the reallocation process is started only after the required authority approval.

5.2.4 Rescheduling

The rescheduling function applies the **Disruption-Aware Dynamic Schedule Adaptation Algorithm** to the affected turns of an approved disruption.

The process does not regenerate the entire monthly schedule. Instead, it works with the existing assignments and modifies only the future turns that are affected by the unavailable bus.

For each affected turn, the system first considers the required preparation time. It then retrieves possible replacement buses and evaluates their eligibility.

A candidate bus is considered only when it satisfies the required operational conditions, including:

- Availability during the required time;
- Suitable duty status;
- No conflicting assignment;
- Sufficient turnaround time; and
- Suitability for the affected service.

Buses that fail these conditions are excluded from the selection process. The remaining candidates are passed to the bus-ranking function.

If a suitable replacement bus and driver can be identified, the affected turn is updated. If the required resources cannot be found, the system cancels the affected turn and records the reason.

5.2.5 Bus Ranking

The bus-ranking function determines which eligible bus should be used when more than one replacement is available.

The selection follows the policy defined for fair and operationally suitable resource allocation. **Route compatibility** is considered first because a bus that is already suitable for the required route is preferable. The system then considers the number of turns assigned to each candidate during the relevant period. Candidates with a lower workload receive higher priority.

The system also considers the recent reallocation history of candidate buses. This reduces the possibility of repeatedly assigning disruption-related work to the same bus.

The ranking order can therefore be expressed as:

Route Compatibility → Lower Daily Workload → Fewer Recent Reallocations

The highest-ranked eligible bus is selected as the replacement. If no candidate satisfies the required conditions, the affected turn proceeds to the cancellation path.

This rule-based selection makes the replacement decision consistent with the defined scheduling policy and allows the reason for a selection to be examined during testing.

5.2.6 Schedule Update

he schedule-update function applies the final decision produced by the rescheduling process to the operational schedule.

When a replacement bus and suitable driver are available, the affected turn is updated with the new resource information. Its status is changed to REALLOCATED, and the relevant change information is retained for traceability.

If the system cannot identify a feasible replacement, the turn is marked as CANCELLED. The reason for the cancellation is stored so that the final scheduling outcome can be reviewed later.

The updated assignment is then persisted in the central PostgreSQL database. Consequently, subsequent schedule requests return the latest state of the affected turn rather than the original assignment.

5.2.7 Notification

The notification function communicates the outcome of the scheduling process to the relevant application interfaces.

When a turn is successfully reallocated, the updated bus, driver, and service information can be presented through the relevant interface. When a turn is cancelled, the system provides the cancellation status together with the recorded reason.

The notification stage does not make the scheduling decision. Its purpose is to communicate the result generated by the scheduling services so that the updated operational state can be viewed by the relevant users.

Overall, these functional units form a sequential scheduling process in which the initial timetable is converted into a monthly operational schedule, approved disruptions are analysed, eligible resources are evaluated, and only the affected future turns are modified. This structure supports the implementation of the Policy-Driven Bus Scheduling and Time-Slot Management component while keeping each major scheduling function separately testable.

5.3 Integration with the other components

The scheduling component operates as part of the wider smart bus system and interacts with the Driver App, Authority Web App, backend services, database, and other functional components. The disruption-handling process is initiated through the Driver App rather than being automatically identified by the IoT monitoring component.

When a bus experiences an operational disruption, the driver submits a disruption request through the Driver App. The request is then received by the backend and made available to the authorised authority user through the Authority Web App. The authority user reviews the request and decides whether to approve or reject the disruption.

The dynamic reallocation process is initiated only after the disruption request is approved. Once approved, the scheduling component identifies the affected future turns and applies the Disruption-Aware Dynamic Schedule Adaptation Algorithm to determine whether suitable replacement resources are available.

The main integration flow is:

Driver App → Disruption Request → Backend → Authority Web App → Authority Approval → Scheduling Component → Dynamic Reallocation → Updated Schedule

This approval-based flow ensures that a scheduling change is not automatically performed based only on a reported disruption. The authority user provides the required operational confirmation before the existing schedule is modified.

5.3.1 Integration Flow

The disruption-handling workflow begins when the driver reports an operational disruption through the Driver App. The driver provides the information required to identify the affected bus and submit the disruption request.

The request is stored and passed to the backend services, where it becomes available to the authorized authority user through the Authority Web App. The authority user reviews the request and either approves or rejects it.

If the request is rejected, no dynamic schedule reallocation is performed. If the request is approved, the scheduling component retrieves the affected bus's future turns within the applicable reallocation window and starts the rescheduling process.

The scheduling component then checks the affected turns and identifies eligible replacement buses based on availability, assignment conflicts, turnaround requirements, and other defined scheduling constraints. Eligible buses are ranked according to the scheduling policy, and an appropriate driver is then selected.

If suitable resources are available, the affected turn is updated with the replacement bus and driver and its status is changed to REALLOCATED. If no feasible replacement is available, the affected turn is changed to CANCELLED and the reason is recorded.

The overall integration flow is therefore:

Driver App → Disruption Request → Authority Web App → Approval → Affected Turns → Candidate Bus Selection → Driver Selection → Schedule Update

The IoT monitoring component is not responsible for identifying or initiating the disruption request in this workflow. Its role remains separate from the authority-controlled disruption and reallocation process.

5.3.2 API and Service Integration

The scheduling component is implemented as a set of services within the Spring Boot backend and communicates with the Driver App and Authority Web App through REST APIs.

The Driver App uses the relevant API to submit a disruption request. The backend receives and stores the request so that it can be reviewed through the Authority Web App. The authority user can then approve or reject the request through the corresponding authority interface.

The reallocation process is not triggered when the driver initially submits the request. Instead, the approval result is passed to the scheduling services. When the request is approved, the disruption handling service identifies the affected bus and determines the applicable reallocation window.

The affected turns are then passed to the schedule adaptation service, which applies the defined scheduling rules to identify feasible replacements.

The service interaction follows the implemented sequence:

Driver App → Disruption API → Disruption Request → Authority Web App → Approval → Disruption Handling Service → Schedule Adaptation Service → Bus Ranking and Driver Selection → Schedule Update

After approval, the schedule adaptation service checks the preparation-time requirement and filters candidate buses based on availability, duty status, assignment conflicts, and turnaround time. The remaining candidates are ranked according to route compatibility, daily workload, and recent reallocation frequency.

After a replacement bus is selected, the driver selection service identifies the regular driver of that bus or an alternative available driver when required.

If both a suitable bus and driver are available, the schedule update service changes the affected turn to REALLOCATED and stores the updated assignment. If no feasible replacement is available, the turn is changed to CANCELLED, and the cancellation reason is recorded.

This service-based structure separates the reporting and approval process from the actual scheduling decision. The Driver App is responsible for reporting the disruption, the Authority Web App provides the approval decision, and the scheduling services perform the resource evaluation and schedule adaptation after approval.

5.3.3 Database Integration

The scheduling and disruption-handling services use PostgreSQL as the central database for storing and retrieving the operational information required by the system.

The database stores information related to buses, drivers, routes, trips, timetable information, schedule assignments, disruption requests, approval status, and reallocation or cancellation results.

When a driver submits a disruption through the Driver App, the disruption request is stored in the database with its relevant information and initial status. The Authority

Web App retrieves this information so that the authorized authority user can review the request.

The approval decision is then stored and used by the backend to determine whether the scheduling process should continue.

For an approved disruption, the scheduling service retrieves the affected bus and its future schedule assignments from the database. Candidate bus and driver information is also retrieved to perform the eligibility and ranking process.

When a turn is successfully reallocated, the updated bus and driver assignment is stored in the database, and the turn status is changed to REALLOCATED. If no feasible replacement is available, the status is changed to CANCELLED, and the cancellation reason is stored.

The database therefore provides the central state required to maintain consistency between the disruption request, authority decision, scheduling process, and final operational schedule.

5.3.4 Integration with IoT and Hardware Components

The wider system includes IoT-based monitoring using ESP32 devices and GPS/IMU-related sensing. However, the IoT component is not used to identify or initiate the disruption request in the implemented scheduling workflow.

Disruptions relevant to schedule reallocation are reported manually by the driver through the Driver App. The request is subsequently reviewed and approved by the authority user before the scheduling component modifies the existing schedule.

Therefore, the IoT monitoring component and the scheduling component have separate responsibilities. The IoT layer supports the wider operational monitoring functions of the system, while disruption reporting and authority-controlled schedule reallocation are handled through the Driver App, Authority Web App, and backend scheduling services.

This separation prevents an automatically detected or sensor-related event from directly changing the operational schedule without the required authority approval.

5.3.5 Interaction with Other Functional Components

The scheduling component also interacts with driver-related information and downstream revenue-related functions within the wider system.

The Driver App is the starting point for the disruption workflow because the driver reports the operational problem and submits the disruption request. The Authority Web App then provides the interface for reviewing and approving the request.

After approval, the scheduling component uses the available bus and driver information to perform the reallocation process. Driver availability and assignment information are considered when selecting a driver for the replacement bus.

The final scheduling status can also be used by the revenue management component as downstream operational information. A REALLOCATED or CANCELLED turn represents the final state of the affected service and can therefore be used by other system functions where required.

The scheduling component therefore acts as the link between **driver-reported disruptions, authority approval, resource allocation, and the updated operational schedule**, while the IoT monitoring component remains separate from the disruption approval and reallocation trigger.

5.4 Verification and software testing

5.4.1 Unit Testing

JUnit and Mockito were used to test the decision rules in isolation, with repositories replaced by mocks so that each rule could be exercised with controlled data. The tested rules were:

- Preparation-time check;
- Bus availability and off-duty check;
- Assignment conflict detection;
- Turnaround validation;
- Route compatibility ranking;

- Daily workload ranking;
- Recent reallocation ranking;
- Driver selection, with and without an available regular driver.

5.4.2 Integration Testing

Integration testing checked that data moves correctly between modules. Postman was used to call the REST endpoints and inspect the responses and the stored records. The tested connections were:

- driver mobile application → authority web portal → scheduling engine;
- disruption event → affected trips;
- affected trips → candidate buses;
- candidate buses → selected bus;
- selected bus → selected driver;
- successful reallocation → notification;
- failed selection → cancellation with stored reason.

5.4.3 System and End-to-End Testing

End-to-end testing started from a generated monthly base schedule and a disruption request raised through the driver mobile application. The test followed the complete trace: original schedule, disruption request, affected trips, candidate buses, selected replacement, selected driver, updated schedule, and the alert received in the driver mobile application. The aim was to confirm that one request leads to a consistent updated schedule without manual steps between the portal and the scheduling engine.

5.4.4 Test Scenarios

The scenarios were defined before the experiments, so each has an expected outcome against which the system decision can be compared.

ID	Scenario	Expected outcome
T01	Monthly schedule creation	Schedule generated
T02	No disruption	Schedule unchanged
T03	Emergency bus unavailable	Trips in the next 24 hours processed
T04	Specific-date disruption	Only trips in the requested range processed
T05	Insufficient preparation time	Turn Cancelled
T06	Candidate bus has an assignment conflict	Candidate rejected
T07	Candidate bus has insufficient turnaround time	Candidate rejected
T08	Same-route buses available	Same-route bus preferred
T09	Equal route compatibility	Lower daily workload preferred
T10	Equal daily workload	Fewer recent reallocations preferred
T11	Regular driver available	Regular driver selected
T12	No replacement bus	Turn cancelled
T13	Successful reallocation	Updated schedule and notifications produced

Table 8: Test Scenarios and Expected Outcomes

5.4.5 User Acceptance and Performance Testing

User acceptance testing is directed at the people who use the schedule: authority staff, bus owners, and drivers. Participants are asked about the clarity of the updated assignments, the usefulness of replacement bus information and cancellation reasons, and the transparency of the selection decisions.

Performance testing measures the time taken to resolve affected trips and how that time changes as the number of affected trips and the size of the fleet grow. Participant feedback and measured response times.

5.4.6 Deployment and Operation

The scheduling services run inside the Spring Boot application and store their data in PostgreSQL.

During operation, a driver sends a disruption request through the mobile application, and the authority web portal passes it to the scheduling engine. The adaptation service produces reallocations and cancellations, and the portals display the updated schedule. The driver whose turn is reallocated receives an alert in the mobile application.

Because every decision and its reason are stored, authorized users can review afterwards how each change was made.

5.5 Deployment and operation

The scheduling component was deployed as part of the wider web-based smart bus system. The Spring Boot backend provides the scheduling services and REST APIs, while the React application provides the user-facing interfaces.

The PostgreSQL database acts as the central data storage layer for operational scheduling information. The backend communicates with the database to retrieve existing schedules, process scheduling decisions, and persist updated assignments.

The deployment structure can be represented as:

React Application → Spring Boot Backend → Scheduling Services → PostgreSQL Database

The wider system additionally contains the IoT monitoring layer, which provides operational information that can be used by the backend when relevant to scheduling decisions.

The deployment separates the presentation, application, scheduling, and data-storage responsibilities. This allows the scheduling logic to operate independently of the user interface while maintaining access to the shared operational data.

The implemented deployment supports the two main operating modes of the scheduling component. During normal operation, the monthly scheduling services generate and maintain the planned schedule. During disruption conditions, the dynamic scheduling services modify only the affected future turns according to the defined policy rules.

All schedule changes are persisted in the central database so that the latest schedule state can be retrieved by the relevant interfaces and used by other system components.

Overall, the deployment provides the required environment for executing the Policy-Driven Bus Scheduling and Time-Slot Management component as part of the wider IoT-enabled smart bus system.

6 RESULTS AND VALIDATION

6.1 Research results

The evaluation covers the whole path from a planned schedule to an operational one. Monthly base scheduling is checked first, because every later decision depends on its output. Reallocation is checked second.

6.1.1 Reference Configuration for Monthly Base Scheduling

The number of assignments in a month depends on the daily timetable and the number of days, not on the number of buses. A reference configuration makes the expected outcome easy to state.

Item	Reference
Trips in the daily timetable	20
Days in the month	30
Active buses	10
Expected assignments in the month	20 × 30 = 600
Expected assignments per bus under an even spread	600/10 = 60

Table 9: Reference Configuration for Monthly Base Scheduling

The even spread of 60 trips per bus is the benchmark against which fairness is judged. It is a target set by the arithmetic of the configuration. How close the generated schedule comes to it is something the experiment has to show.

6.1.2 Evaluation Questions

Seven questions guide the evaluation.

- Does the monthly base schedule contain the expected number of assignments?
- Are trips spread evenly across the active buses?
- Do all assignments respect the no-overlap rule and the 15-minute turnaround time?
- Are affected trips identified correctly for the reallocation window in use?

- Are unsuitable buses rejected, and do the selected bus and driver follow the stated order of rules?
- Are feasible trips reallocated and infeasible trips cancelled with a stored reason?
- How long does the decision take for each affected trip?

6.1.3 Evaluation Measures

Measure	**Definition**
Assignment completeness	Assignments generated compared with trips per day multiplied by days in the month
Workload spread	Difference between the highest and lowest number of assigned trips among active buses
Constraint violations in the base schedule	Overlapping assignments and turnaround breaches found
Reallocation success rate	Reallocated trips as a percentage of affected trips
Cancellation rate	Cancelled trips as a percentage of affected trips
Constraint compliance rate	Decisions that respect every constraint as a percentage of all decisions
Average processing time	Mean time taken to resolve one affected trip

Table 10: Evaluation Measures and Definitions

6.2 Product and Technical Results

6.2.1 Monthly Base Scheduling

The generated schedule is examined with the checks below.

Check	What is verified	Expected outcome
Completeness	Count of stored assignments	600 assignments for the reference configuration
No overlap	No bus holds two trips whose times overlap	Zero overlapping assignments
Turnaround	Gap between the arrival of one trip and the departure of the next trip of the same bus	At least 15 minutes in every case
Active status	Buses marked inactive	No assignment given to an inactive bus
Continuous turn preference	Choice between buses with equal trip counts	The bus whose previous trip ended where the next trip starts is preferred
Deterministic tie-break	Choice between buses equal on every criterion	The lower bus ID is chosen, and two runs on the same input give the same schedule

Table 11: Verification Checks on the Generated Monthly Base Schedule

The evaluation reads the schedule day by day, in departure order, which is the order in which the algorithm itself works. For every trip, eligibility is checked first (active status and a turnaround of at least 15 minutes after the previous arrival). Among eligible buses, the one with the fewest trips so far in the month is chosen. Equal counts are settled by continuous turn preference, and any remaining tie by bus ID.

6.2.2 Disruption Handling and Reallocation

The reallocation tests use the scenarios T01 to T13 from Chapter 5. Each scenario has an expected decision that is written down before the run.

Evaluation area	What is checked	Scenarios	Outcome reported
Affected trip identification	Correct window, correct turns	T03, T04	Affected trips found against affected trips expected
Preparation-time validation	Early cancellation before any search	T05	Cancellations produced against cases that should cancel
Candidate bus filtering	Conflict and turnaround rejection	T06, T07	Candidates rejected against candidates that should be rejected
Bus ranking	Route compatibility, then daily workload, then recent reallocations	T08, T09, T10	Cases in which the selected bus matches the expected bus
Driver selection	Regular driver first, lowest daily workload as fallback	T11	Cases in which the selected driver matches the expected driver
Cancellation	No eligible bus, or no suitable driver	T12	Cancellations with a stored reason

Reallocation	Updated turn, status, and notification	T13	Reallocation success rate and notifications generated
Processing time	Time to resolve one affected trip	All scenarios	Average processing time

Table 12: Evaluation Results for Disruption Handling and Reallocation

A scenario counts as passed only when the decision, the stored status, and the stored reason all agree with the expectation. A correct bus with no recorded reason is not a pass, because traceability is a requirement on its own (NFR2).

6.3 User or stakeholder validation

A schedule is useful only if the people who read it can understand and act on it. The stakeholders are authority staff, bus owners, and drivers.

The validation looks at six points:

- How clear is the monthly summary of trips per bus?
- How clear the updated assignments are after reallocation;
- How useful the replacement bus and driver information is;
- How useful the stored cancellation reasons are;
- How understandable the error messages are when generation cannot proceed;
- How much manual scheduling effort participants feel is removed.

Participant numbers, roles, and responses are reported only for sessions that were actually held with the required approval. Where no session took place, no claim about stakeholder opinion is made.

6.4 Integration results

Integration testing validates requests across the entire system pipeline, tracing data flow from the user interface down to the persistent schedule storage.

6.4.1 Monthly Schedule Generation Path

A user selects the route, timetable, and target month (e.g., `2026-09`) in the monthly schedule generator on the React interface and triggers generation. The request is dispatched to the `/api/schedules/monthly-generate` endpoint of the Spring Boot backend. The scheduling service iterates through every day of the selected month, executes eligibility validation and fair bus-driver pair selection logic for each trip, and persists every result as a schedule assignment. The monthly summary view subsequently displays the total scheduled trips and the allocation count per bus unit.

Integration checks compare the state rendered in the summary view against the records saved in the database, verifying that valid requests match expected operational totals. Edge cases are evaluated concurrently: if a generation request is initiated when no active buses exist, the system safely aborts execution, prevents partial schedule persistence, and returns an explicit, actionable error message.

6.4.2 Disruption and Reallocation Path

When a service disruption occurs, the operator/driver submits an incident report via the mobile application. The transit authority portal captures the event and routes it to the dynamic scheduling engine. Since a bus and its assigned driver operate as a unified resource unit, declaring a bus unavailable automatically marks its associated driver as unavailable.

The engine then:

1. Identifies all impacted turns.
2. Filters out candidate bus-driver units rendered unavailable or in operational conflict.
3. Ranks remaining eligible bus-driver units via the fair decision matrix (considering route match and accumulated daily workload).
4. Reassigns the turn to the top-ranked eligible unit.
5. Updates the schedule assignment and fires real-time alerts.

Both authority and operator mobile applications automatically update, and the crew of the newly assigned bus receives an immediate notification. If no feasible bus-driver unit is available, the turn status is set to CANCELLED, the exact operational reason is recorded, and the UI reflects the cancellation accordingly.

Every integration path satisfies the core integrity invariant: **Each turn holds exactly one valid bus-driver assignment, no resource unit is committed to overlapping turns, and every stored record retains its decision rationale.**

6.4.3 Worked Decision Trace

Consider a scenario where Bus Unit B01 (Bus B01 + Driver D01) holds turns at 08:00, 10:00, 12:00, and 14:00, and reports an unexpected breakdown/unavailability at 09:30. The trace below illustrates the expected execution of the decision engine:

Affected Trip	Decision Path	Expected Result
10:00	Preparation time is insufficient Remaining Time < MIN_PREPARATION_TIME).	Cancelled; operational reason stored.
12:00	Unit B02 (with Driver D02) rejected due to schedule conflict. Units B03 (Driver D03) and B04 (Driver D04) are eligible. Unit B03 ranks first due to primary route match and lower daily workload (3 turns vs 5 turns).	Reallocated to Unit B03 (Bus B03 + Driver D03).
14:00	Processed independently using the identical evaluation sequence.	Assigned independently based on real-time unit availability at execution.

Table 13: Worked Decision Trace for Bus Unit B01 Breakdown

6.5 Summary of findings

The findings are read against the research questions of Chapter 1.

Research question	**Evidence that answers it**
RQ1: structuring the monthly schedule so turns can be managed	Completeness, workload distribution, and the turn records stored after generation
RQ2: building constraints into replacement bus selection	Overlap and turnaround checks, scenarios T05 to T07, and the stored rejection reasons
RQ3: fair allocation through workload and recent reallocations	Workload spread in the base schedule, scenarios T08 to T10, and workload distribution before and after disruption handling
RQ4: schedule continuity when a bus becomes unavailable	Reallocation success rate and constraint compliance rate across all scenarios
RQ5: when to reallocate and when to cancel	scenarios T05, T11, and T12 compared with T13

Table 14: Summary of Findings against the Research Questions

7 DISCUSSION

An analysis of the Policy-Driven Bus Scheduling and Time-Slot Management system is presented in Chapter 7, including a comparison of the design to what has come before, the contributions made by both the research and the product, and the limits of the claims being made.

7.1 Interpretation of findings

A component of this nature generates decisions, rather than predictions. All outcomes are direct operational actions, namely that a trip is assigned to a bus, a turn is allocated to another bus and driver, or a turn is canceled for a given reason. Therefore, the results that are analyzed in Chapter 6 should be treated as evaluations of the validity, fairness, and timeliness of these actions rather than their accuracy.

7.1.1 What the Results Say About the Design

The monthly base schedule is the foundation of everything that follows. Its size is fixed by the daily timetable and the number of days, not by the number of buses, so a timetable of 20 trips over 30 days always yields 600 assignments. Fairness is then a question of how those 600 assignments are divided. Because the algorithm always gives the next trip to the eligible bus with the fewest trips so far, the division stays close to even by construction, and Jain's Fairness Index gives a single number for how close it came.

Determinism shapes how every result should be read. The same input always produces the same schedule and the same reallocation decision, so a result is a property of the rules and the data, not of luck. A different outcome on a second run would point to a defect, not to variation.

7.1.2 Reading the Metrics Together

No single measure carries the evaluation.

- A high successful reallocation rate (SRR) is good only when the constraint compliance rate (CCR) is also high. A system that reallocates every trip by ignoring conflicts would score well on the first measure and fail on the second.
- A non-zero cancellation rate (CR) is not a failure. When preparation time is too short, or no eligible bus or driver exists, cancellation is the correct decision, and the design treats it as a controlled outcome with a stored reason.
- A good fairness index at planning time says little about fairness after a disruption event. The workload distribution before and after reallocation shows whether the recent reallocation criterion actually keeps the extra trips from piling onto one bus.
- Execution time matters only relative to the moment of use. A decision that arrives after the departure time has no operational value, however correct it is.

7.1.3 Answering the Research Questions

The findings are used to answer the questions posed in Chapter 1.

Research question	**How the findings address it**
RQ1: Structuring the monthly schedule for turn-level management	Storing each assignment as a turn with its own status makes individual trips addressable and changeable without regenerating the month
RQ2: Building constraints into replacement bus selection	Candidate bus filtering removes unavailable, off-duty, conflicting, and turnaround-short buses before ranking, so constraint handling is separate from preference handling
RQ3: Fair allocation through workload and recent reallocations	Ranking by route compatibility, then daily workload, then recent reallocations

	spreads replacement work; the fairness index and the before-and-after workload distribution measure it
RQ4: Schedule continuity after a bus becomes unavailable	SRR together with CCR shows how much of the affected schedule survives while staying valid
RQ5: when to reallocate and when to cancel	Cancellation follows three gates: insufficient preparation time, no eligible bus, and no suitable driver; every other trip is reallocated

Table 15: Answers to Research Questions

7.2 Comparison with literature baselines and existing solutions

Research on public transport rescheduling is extensive. Lai and Leung studied dynamic vehicle and crew scheduling under uncertain, time-dependent travel conditions with a rolling-horizon approach [1]. Reviews of bus operations list holding, rescheduling, and stop-skipping among the main control actions against disruption [2]. GTFS separates static schedule information from real-time updates [6], [7], which matches the split in this component between the monthly base schedule and a disruption event.

The comparison below is conceptual. It sets the approaches side by side on the properties that matter for this component.

Aspect	**Manual Process**	**First available bus**	**Optimization-based rescheduling [1]**	**Proposed component**
Monthly planning	Assigned by hand	First eligible bus found	Handled inside the joint vehicle and crew model	Fewest trips so far among eligible buses, then continuous turn, then bus ID

Basis of decision	Operator's judgement	Availability flag	Mathematical objective	Explicit constraints and ranked priorities
Handling of conflicts and turnaround	Checked by hand	Not checked	Modelled as constraints	Checked in candidate bus filtering and in the base schedule
Fairness	Informal	None	Depends on the objective	Trip count in planning; daily workload and recent reallocations in reallocation
Failure handling	Manual decision	Trip left unresolved	Infeasible model	Controlled cancellation with a stored reason
Explainability	Depends on the person	Trivial but shallow	Low to moderate	High; each decision follows a named rule
Parameter and computing demand	Low	Low	High	Low

Table 16: Conceptual Comparison of the Proposed Component with Existing Approaches

Three points follow from the table. First, the component trades global optimality for traceability and light computation, so an operator can see why a bus was chosen or

rejected, which an optimization model rarely shows. Second, fairness is handled at both stages: by trip count when the month is planned, and by daily workload and recent reallocations when a trip has to be replaced. Third, the component covers the whole path from planning through reallocation to cancellation in one workflow, where the cited work concentrates on the rescheduling model itself.

The component was not benchmarked numerically against the optimisation methods in [1]. The comparison therefore supports a claim about design properties and not about relative solution quality.

7.3 Research contribution

The contribution is an integrated rule-based workflow that joins planning and disruption response, which are usually treated as separate problems. It has two parts.

Monthly fair scheduling

1. **Planning independent of fleet size.** The number of assignments follows from the timetable and the calendar, not from the number of buses.
2. **Fair allocation under hard constraints.** Each trip goes to an eligible bus with the fewest trips so far, with no overlap, a 15-minute turnaround, and active status enforced.
3. **Ordered tie-breaking.** Continuous turn preference comes first and the bus ID last, which makes the schedule reproducible.
4. **Turn-level representation.** Every assignment is stored as a turn with its own status, so single trips can change later without regenerating the month.

Reallocation after a disruption event

5. **Trigger-based adaptation.** A driver request opens a reallocation window of 24 hours for emergencies, or a requested range for specific-date operations, and only the affected trips inside it are processed.
6. **Separation of filtering from ranking.** Hard constraints remove candidates first, and preferences order the survivors, so a constraint is never traded against a preference.

7. **Controlled cancellation.** A trip with no feasible replacement is cancelled with a stored reason, so the updated schedule never contains an invalid assignment.

Taken together, these steps describe a path from planned scheduling to adapted operation that can be explained at every stage.

7.4 Product contribution

The product contribution is the working component inside the wider smart bus platform.

- A monthly schedule generator in the React interface, backed by a Spring Boot endpoint, that stores every assignment and shows the total and per-bus trip counts.
- A turn repository in PostgreSQL that keeps the original assignment and the status changes side by side.
- A disruption path in which a driver sends a request through the mobile application, the authority web portal receives it, and the scheduling engine processes it.
- Automatic bus filtering and ranking, driver selection, reallocation, and cancellation with stored reasons.
- Notifications to stakeholders and an updated schedule visible in the authority, owner, and driver interfaces.
- Configurable policy values, such as minimum preparation time and turnaround time, that can change without altering the algorithm.

7.5 Limitations and threats to validity

7.5.1 Limitations of the Component

- **Dependence on accurate data.** Bus status, driver availability, trip times, and current assignments must be correct and current. A stale record produces a decision that is correct for the data and wrong for the road.

- **Dependence on configured rules.** Thresholds such as minimum preparation time, the 15-minute turnaround, and the order of ranking criteria are set by policy. Different values give different decisions.
- **One turnaround value for every trip.** The monthly schedule applies a single turnaround time, so routes that need more or less time between trips are not distinguished.
- **Constraint-limited fairness.** At busy times few buses may qualify, which can leave the monthly split slightly uneven even though the rule always favours the least loaded bus.
- **Plan fixed at generation.** A change to the timetable or the active fleet after generation means generating the month again or handling the change through reallocation.
- **Driver-initiated trigger.** Disruption events enter through a driver request. The component cannot react to a breakdown that nobody reports, and the reallocation window starts only when the request arrives.
- **Workload measured by trip count.** Fairness counts trips and does not weigh trip length, distance, or hours worked.
- **Strict preparation-time gate.** A trip with too little preparation time is cancelled before any search, even when a bus might technically have become free later.
- **Single-disruption focus.** Several simultaneous breakdowns affecting interconnected schedules may need more than independent, trip-by-trip processing.
- **No demand awareness.** Decisions rest on operational feasibility and not on passenger demand.
- **Feasible, not optimal.** The algorithm finds a valid and fair assignment according to its rules. It does not claim to find the best assignment for the whole network.

7.5.2 Threats to Validity

Threat	Description	Handling
Internal	Test scenarios are written by the same person who designed the rules, so they may favor the design	Expected outcomes are written before each run; scenario coverage includes failure cases such as T12
Construct	SRR, CCR, and the fairness index measure validity and balance, not passenger or operator satisfaction	Stakeholder feedback is reported separately and only when sessions were actually held
External	Simulated or limited operational data may not represent the conditions of a real fleet	Findings are stated for the data used; broader use needs real operational records
Conclusion	A small number of scenarios cannot establish performance at scale	Execution time is examined as the number of affected trips and the fleet size grow

Table 17: Threats to Validity and Their Handling

7.6 Implications

For bus operators. Replacement decisions become faster, more consistent, and easier to defend. A timekeeper no longer must search for a bus, check its previous trip, and find a driver by hand, and every decision leaves a record that can be reviewed afterwards.

For transport authorities. Policy values live in configuration, so an authority can adjust preparation time, turnaround time, and priorities without changing the program. The stored reasons give a basis for auditing how disruptions were handled.

For research. The component offers an explainable rule-based reference point. Later work can compare optimization or heuristic methods against it on the same scenarios, which would show what the extra complexity buys.

For deployment. Wider use depends on validation with real operational data, authority-specific policies, larger fleets, and simultaneous disruptions, as Chapter 9 describes. Until then, the component is best understood as a working and well-explained approach to one defined problem, not as a finished replacement for human scheduling judgement.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Conclusion

Bus services run on a plan, and a plan is only useful for as long as its assumptions hold. A monthly schedule assumes that every assigned bus will be available. When one is not, the schedule must change quickly, and it must change without breaking the rules that make a schedule valid. This thesis set out to build a component that handles both halves of that problem: producing a fair monthly schedule in the first place, and adapting it when a bus drops out.

The component is the Policy-Driven Bus Scheduling and Time-Slot Management engine within the IoT-enabled smart bus system. In the wider flow of the system it sits after permit and route assignment by the transport authority and before operational monitoring and revenue calculation.

Planning stage. Once buses are assigned to a route, the engine generates the monthly schedule from the route timetable and the number of days in the month. The number of assignments therefore follows from the timetable and not from the size of the fleet. Each trip is given to an eligible bus, meaning one that is active, does not overlap another trip, and has the required turnaround time. Among eligible buses, the one with the fewest trips so far is chosen, so work is spread evenly across the available buses. Continuous turn preference and the bus ID settle any remaining tie, which keeps the result reproducible.

Disruption stage. When a bus becomes unavailable, the bus owner submits a request through the mobile application. The transport authority reviews it in the web application and accepts or rejects it. After acceptance, the engine marks the bus as unavailable, identifies the affected future trips inside the reallocation window, and reassigns them among the remaining buses on the same route instead of cancelling the whole service. Each trip passes through a preparation-time check, candidate bus filtering, ranking by route compatibility, daily workload, and recent reallocations, and driver selection after the bus has been chosen. A trip for which no feasible bus or driver

exists is cancelled and the reason is stored, so the updated schedule never holds an invalid assignment.

Three features give the approach its character. Decisions follow named rules, so each one can be explained and audited. Filtering is kept apart from ranking, so a constraint is never traded against a preference. And fairness is applied at both stages, once when the month is planned and again when trips are replaced.

The work shows that a rule-based, deterministic design can carry a bus operation from planning through disruption handling in a single workflow that operators and authorities can follow. How well it performs is settled by the experiments in Chapter 6, and the claims of this thesis go no further than the data used there.

8.2 Achievement of Objectives

The table sets each objective from Chapter 1 against the work that addresses it.

Objective	**How it was addressed**	**Where reported**
Design a monthly base scheduling mechanism that allocates planned bus turns to available buses.	Monthly scheduling service that works through each day in departure order and assigns each trip to the eligible bus with the fewest trips so far.	Sections 4.4.2 and 5.2.1; evaluated in 6.2.1
Represent the generated schedule as manageable operational turns.	Turn records holding date, route, departure time, bus, driver, and status, with the status values SCHEDULED, REALLOCATED, and CANCELLED.	Sections 4.4.3 and 5.2.2
Define constraints for bus availability, assignment	Active-status check, overlap check, 15-minute turnaround rule, and minimum	Sections 4.4.5 to 4.4.6 and 5.2.4; evaluated in T05 to T07 and 6.2.1

conflicts, preparation time, and turnaround time.	preparation time applied in the base schedule and in candidate bus filtering.	
Identify the future turns affected by bus unavailability.	Reallocation window (24 hours for emergencies, a requested range for specific-date operations) and retrieval of the affected bus's turns inside it.	Sections 4.4.4 to 4.4.5 and 5.2.3; evaluated in T03 and T04
Rank replacement buses fairly by route compatibility, daily workload, and recent reallocations.	Ranking applied after filtering, with route compatibility first, daily workload second, and recent reallocations third.	Sections 4.4.7 and 5.2.5; evaluated in T08 to T10
Reallocate affected turns, or cancel them safely when no feasible replacement exists.	Three-gate sequence that ends in reallocation or in cancellation with a stored reason, followed by notification.	Sections 4.4.9 and 5.2.4; evaluated in T12 and T13

Table 18: Achievement of Objectives

8.3 Individual Research and Product Contributions

8.3.1 Research Contribution

The research contribution is a rule-based workflow that joins planned scheduling and disruption-aware rescheduling, two problems that are often studied apart. Its distinguishing elements are:

- A monthly schedule whose size depends on the timetable and calendar, allocated by trip count under hard constraints;

- A turn-level representation that lets single trips change without regenerating the month;
- A reallocation window tied to the type of disruption event;
- The separation of constraint filtering from preference ranking;
- Controlled cancellation with stored reasons.

8.3.2 Product Contribution

The product contribution is the working component within the wider system. It includes:

- A monthly schedule generator in the web interface, backed by a Spring Boot endpoint, with a summary of total trips and trips per bus;
- A PostgreSQL turn repository that keeps the planned assignment and every later status change;
- A disruption path from the bus owner's request in the mobile application, through the authority's review in the web application, to the scheduling engine;
- Automatic filtering, ranking, driver selection, reallocation, and cancellation;
- Notifications and an updated schedule visible to the authority, owners, and drivers;
- Policy values that can be configured without changing the algorithm.

8.3.3 Place in the Overall System

The component supplies the other parts of the system with a valid operational schedule. The IoT monitoring component compares actual movement with the approved route, and the driver behavior component reviews driving events. The revenue component assigns time-slot rates and calculates payment from verified distance. All three rely on knowing which bus was meant to run which trip, and the updated schedule from this component answers that question after a disruption.

8.4 Future directions

Several extensions follow from the limitations in Chapter 7.

- **Simultaneous disruptions.** Handle several unavailable buses on the same route or on connected routes, where trips cannot be processed fully independently.
- **Automatic disruption detection.** Connect the IoT monitoring component so that a breakdown can raise a candidate disruption event, which the authority can then confirm.
- **Demand-aware ranking.** Include passenger demand when ranking replacement buses and when recalculating service intervals.
- **Richer fairness measures.** Weigh trip length, distance, or hours worked instead of counting trips alone.
- **Route-specific turnaround.** Replace the single turnaround value with values that depend on the route and the time of day.
- **Traffic-aware turnaround.** Use real-time traffic information when judging whether a replacement bus can reach the departure point.
- **Link to time-slot revenue.** Pass the updated schedule to the revenue component so that reallocated trips are classified and paid under the right time-slot rate.
- **Comparison with optimization methods.** Benchmark the rule-based algorithm against optimization and heuristic approaches on the same scenarios, to show what additional complexity returns.
- **Larger datasets.** Evaluate the component on real operational data, larger fleets, and multi-route networks.
- **Authority-specific policy.** Allow each transport authority to define its own scheduling policy and thresholds.
- **Predictive disruption handling.** Study whether disruption likelihood can be anticipated early enough to prepare reallocation in advance.

REFERENCES

[1] O. J. Ibarra-Rojas, F. Delgado, R. Giesen and J. C. Muñoz, "Planning, operation, and control of bus transport systems: A literature review," *Transportation Research Part B: Methodological,* vol. 77, pp. 38-75, 2015.

[2] A. Ceder, Public transport scheduling, Berlin, Germany: Germany, 2001.

[3] A. Radmankian, A. Pamidimukkala, S. Kermanshachi and J. M. Rosenberger, "Transit network timetabling: A literature review," *Transportation Research Procedia,* vol. 91, p. 750–757, 2025.

[4] V. Boyer, O. J. Ibarra-Rojas and Y. Á. Ríos-Solís, "Vehicle and crew scheduling for flexible bus transportation systems," *Transportation Research Part B: Methodological,* vol. 112, p. 216–229, 2018.

[5] A. Andrade-Michel, Y. A. Ríos-Solís and V. Boyer, "Vehicle and reliable driver scheduling for public bus transportation systems," *Transportation Research Part B: Methodological,* vol. 145, p. 290–301, 2021.

[6] K. Gkiotsalitis, "Bus operations optimization: A literature review on bus holding, rescheduling and stop-skipping," in *Proceedings of the 99th Annual Meeting of the Transportation Research Board*, Washington, DC, USA, 2020.

[7] K. Gkiotsalitis and O. Cats, "At-stop control measures in public transport: Literature review and research agenda," *Transportation Research Part E: Logistics and Transportation Review,* p. 145, 2021.

[8] K. Gkiotsalitis, O. Cats and T. Liu, "A review of public transport transfer synchronisation at the real-time control phase," *Transport Reviews,* vol. 43, p. 88–107, 2023.

[9] D. S. W. Lai and J. M. Y. Leung, "Real-time rescheduling and disruption management for public transit," *Journal of Transportation Engineering, Part A: Systems,* 2017.

[10] S. Zhang, H. K. Lo, K. F. Ng and G. Chen, "Metro system disruption management and substitute bus service: A systematic review and future directions," *Transport Reviews,* vol. 41, no. 2, p. 230–251, 2021.

[11] General Transit Feed Specification, "GTFS Realtime Documentation," [Online]. Available: GTFS Realtime Documentation.

GLOSSARY

Affected trip – A scheduled turn that is assigned to the unavailable bus and falls inside the reallocation window.

Bus unit – A bus paired with its regular driver in the monthly base schedule; the two are treated as separate resources once a disruption is handled.

Candidate bus filtering – The step that removes buses that are unavailable, off duty, already assigned at the same time, or without enough turnaround time before ranking begins.

Cancellation – A controlled fallback in which an affected trip is withdrawn, with the reason stored, when no valid bus or driver can be found or the preparation time is too short.

Daily workload – The number of trips a bus is already assigned on a given day, used as a ranking criterion.

Disruption event – An incident, such as a breakdown or driver unavailability, that makes a scheduled bus unable to complete its assigned trips.

Disruption-aware dynamic schedule adaptation – The rule-based process that identifies affected trips, filters candidate buses, ranks them, selects a driver, and produces an updated schedule after a disruption event.

Driver selection – The step that assigns an available driver to the chosen replacement bus.

Monthly base scheduling – The generation of the planned monthly schedule by assigning each trip to the eligible bus with the fewest assigned trips.

Operational state – The current condition of the schedule and its resources, covering which buses and drivers are assigned, available, or free for the next trip.

Peak and night slots – The time-slot categories used to group bus work and supply policy context for scheduling.

Preparation time – The time remaining between a disruption event and the departure of an affected trip, compared with the minimum preparation time before any search begins.

Reallocation – The replacement of the bus and driver of an affected trip, after which the trip status changes from SCHEDULED to REALLOCATED.

Reallocation window – The time range in which affected trips are identified: 24 hours after the event time for an emergency, or the requested date range for a specific-date operation.

Recent reallocations – The number of replacement assignments a bus has taken recently, used as a tie-breaker so that one bus does not absorb every disruption.

Route compatibility – The degree to which a candidate bus suits the route of the affected trip, used as the first ranking criterion.

Time-slot management – The handling of the time slots of affected trips as an operational resource-allocation problem, beyond simply classifying periods as peak, normal, or night.

Turn – A stored record of one scheduled trip, made up of date, route, departure time, bus, driver, and status.

Turnaround constraint – The requirement that a candidate bus has enough time between finishing its previous assignment and starting the affected trip.

Updated schedule – The schedule after reallocation and cancellation decisions, which contains no invalid or unfulfillable assignments.

APPENDICES

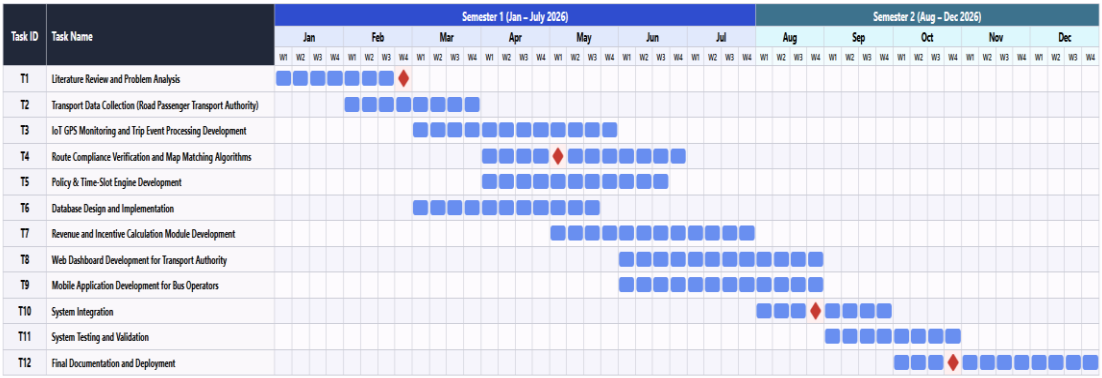
Appendix A: Risk Analysis Matrix

Appendix B: Academic Integrity and AI Use Disclosure

Tool	Purpose	Representative use	Verification performed
ChatGPT (OpenAI)	Learning and conceptual understanding	Used to clarify selected technical concepts related to scheduling algorithms, operational constraints, and disruption handling	Compared the explanations with the implemented component and relevant technical references.
ChatGPT (OpenAI)	Idea exploration and design support	Used to explore alternative approaches for component architecture, scheduling workflows, and diagram layouts.	Evaluated the suggestions against the project requirements and the actual system design.
ChatGPT (OpenAI)	Software development support	Used to understand selected coding issues, debugging approaches, and potential test scenarios.	Tested and reviewed relevant changes against the component's expected behavior.
Claude (Anthropic)	Writing and language refinement	Used to improve grammar, sentence clarity, and wording in selected thesis sections.	Reviewed the suggested edits to preserve the author's intended meaning and technical accuracy.

Appendix C: Gantt Chart

Research Project Gantt Chart — 2026



Key Milestones

- Month 2 – Dataset Preparation Completed
- Month 4 – Core System Components Developed
- Month 8 – System Integration & Testing Completed
- Month 10 – Final Documentation & Deployment

Legend: Task in Progress, Milestone